

METU Computer Engineering

CEng 111 – Fall 2026
Week 4a

Credit: Some slides are from the “Invitation to Computer Science” book by G. M. Schneider, J. L. Gersting and some from the “Digital Design” book by M. M. Mano and M. D. Ciletti.

Previously on CENG111!

Turing Machine



Alan Turing
(1912-1954)

- Follows these steps:
  - Read current symbol on the tape
  - Look at the rule book for that symbol
  - Execute the corresponding rule
- Example (from the second textbook):

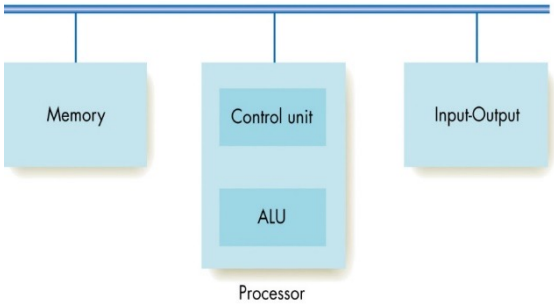
State now	Read symbol	Write symbol	Move tape	Next state
A	0	1	RIGHT	**B**
A	1	1	LEFT	**C**
B	0	1	LEFT	**A**
B	1	1	RIGHT	**B**
C	0	1	LEFT	**B**
C	1	1	RIGHT	**HALT**

METU Computer Engineering

Previously on CENG111!

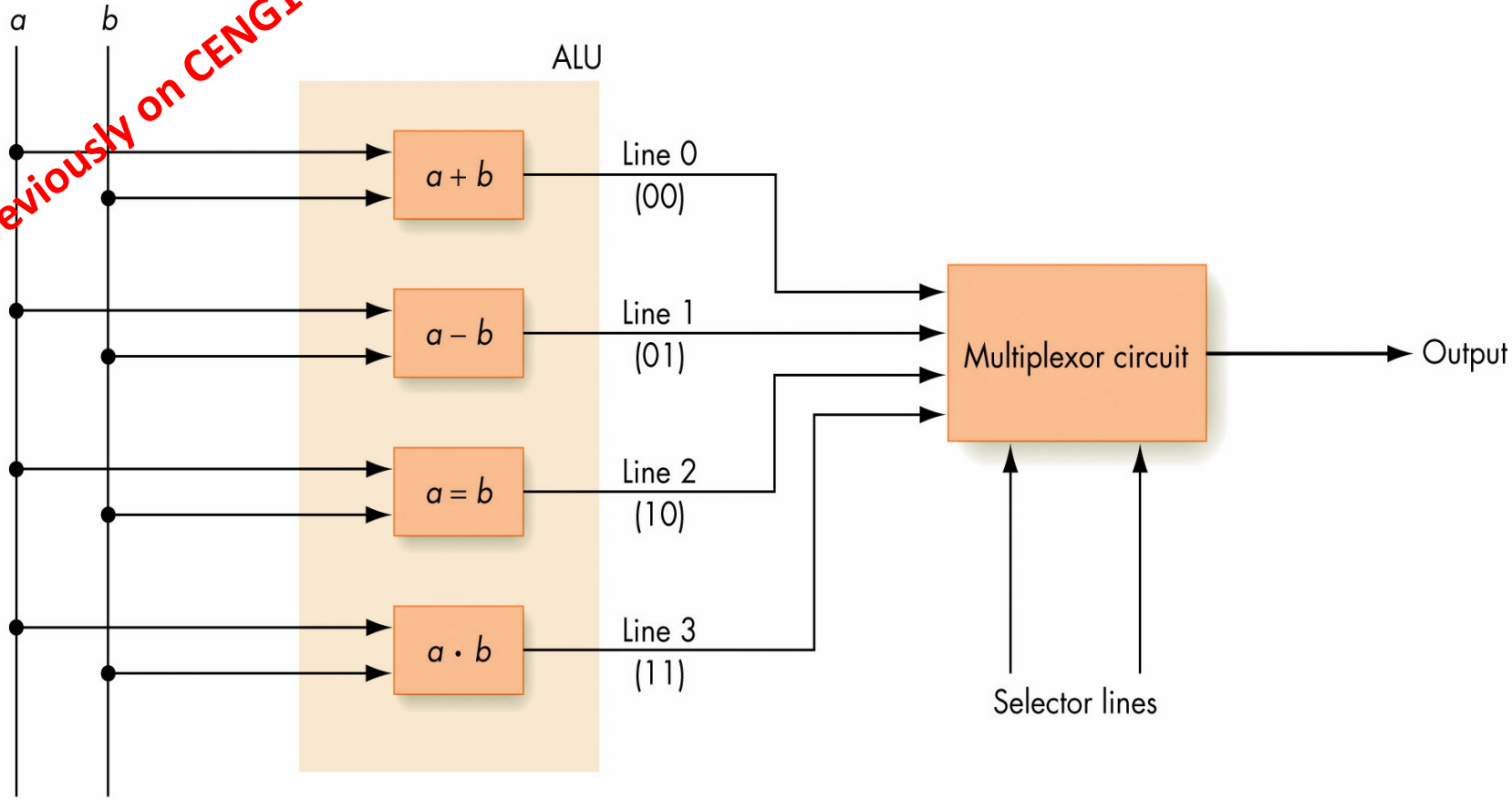
The Arithmetic/Logic Unit

- *Arithmetic and **L**ogic **U**nit*
  - *"Manufacturing" section of computer*
  - *Contains decision mechanisms and can make calculations+comparisons*
  - Actual computations are performed
- Primitive operation circuits
  - Arithmetic *[+, -, *, /]*
  - Comparison *[equality or CE, GT, LT, NEQ]*
  - Logic *[AND, OR, NOT, XOR]*
- Data inputs and results stored in registers
- Multiplexer selects desired output



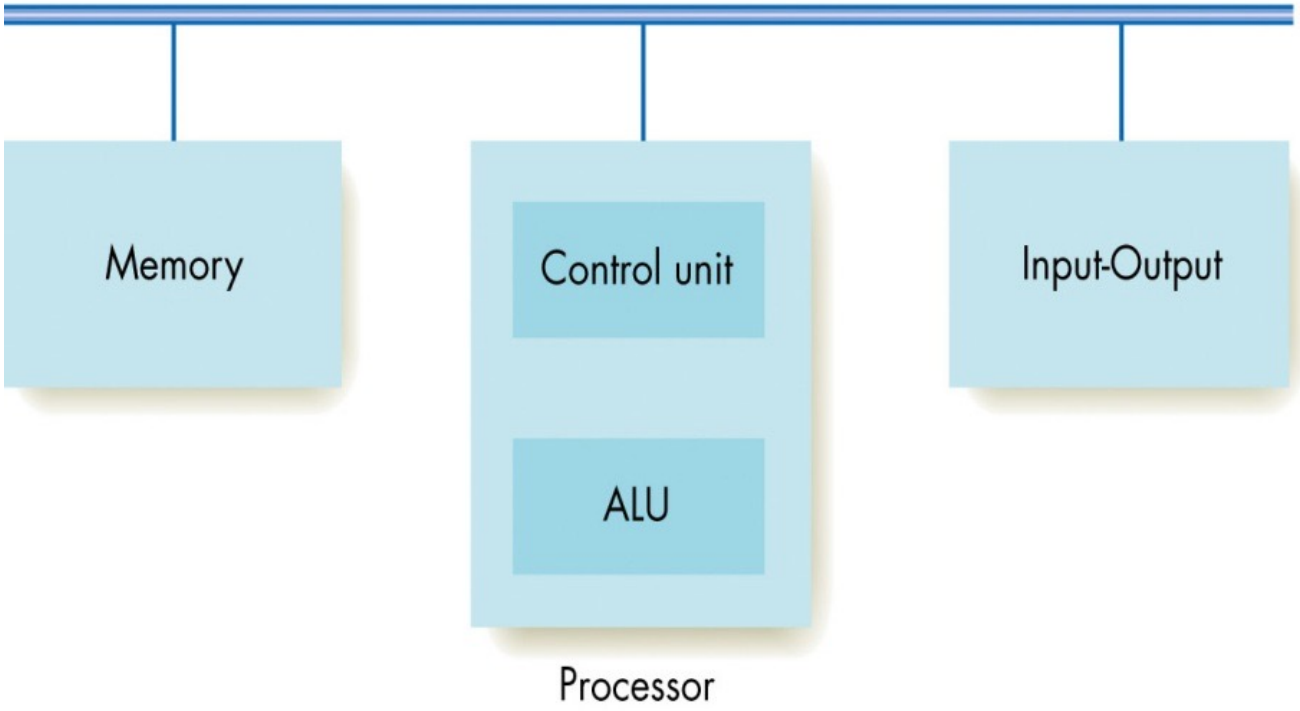
METU Computer Engineering

Previously on CENG111!



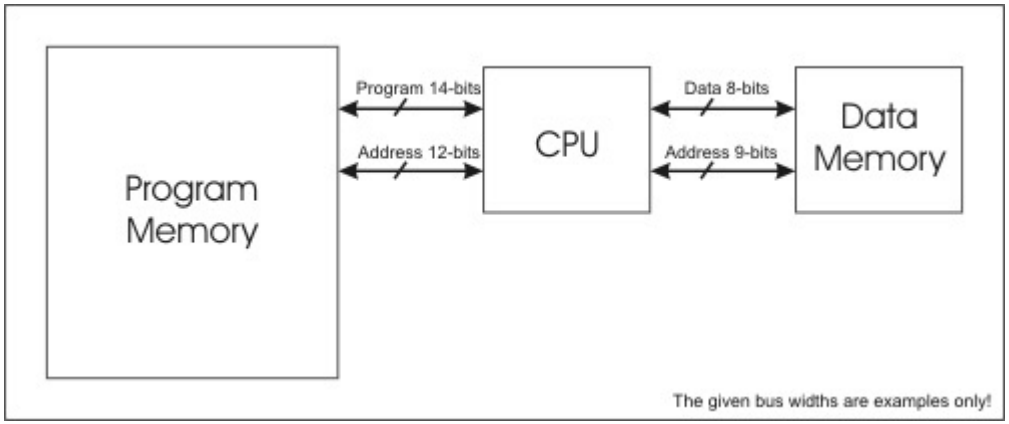
Using a Multiplexor Circuit to Select the Proper ALU Result

Previously on CENG111!



Harvard Architecture

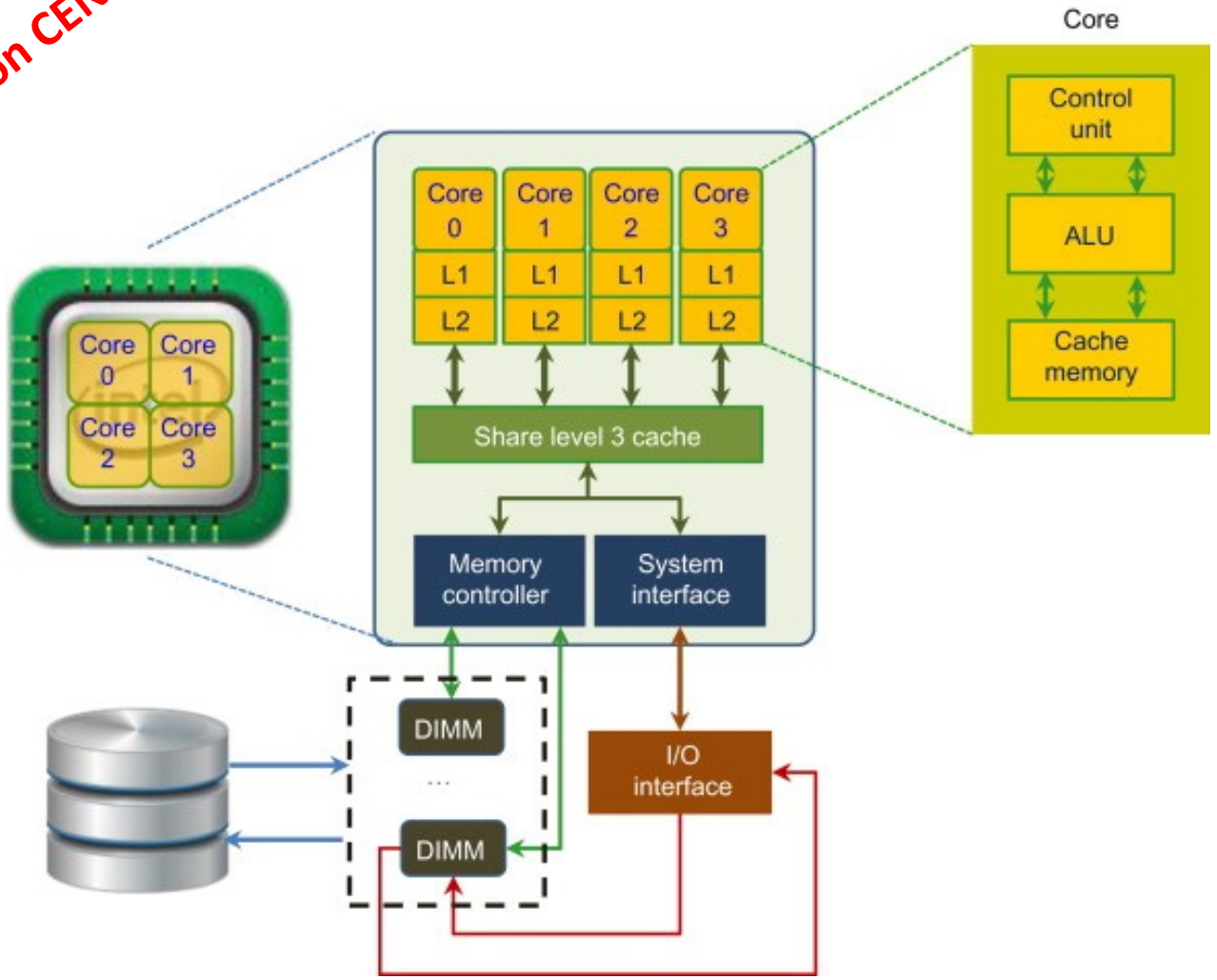
Previously on CENG111!



http://www.mhennessy.f9.co.uk/pic/architecture.htm

- **Pros:**
  - Data and address busses can be of different widths. This means the program memory word can be wide enough to incorporate an instruction and a literal (fixed data) in a single instruction.
  - A built-in two-stage pipeline overlaps fetch and execution of instructions, meaning most instructions execute in a single clock cycle.
- **Cons:**
  - Slightly more confusing.
  - Hardware is more complicated.

Previously on CENG111!

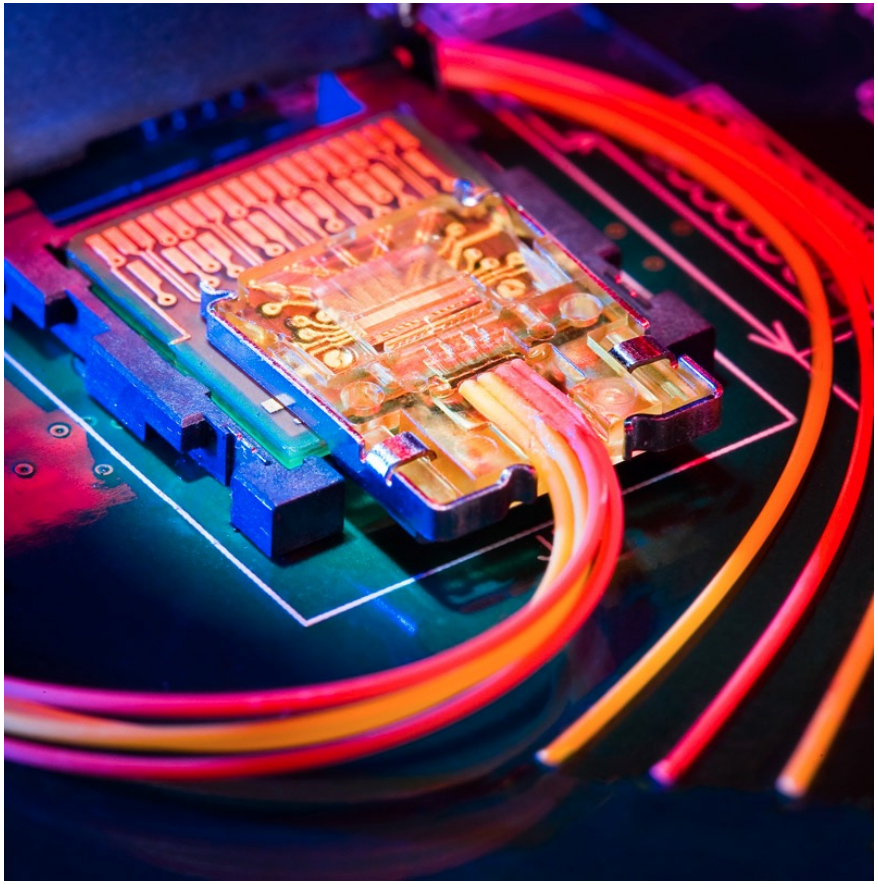


https://www.sciencedirect.com/topics/computer-science/core-processor

Previously on CENG111!

New Trend: Optical Computing

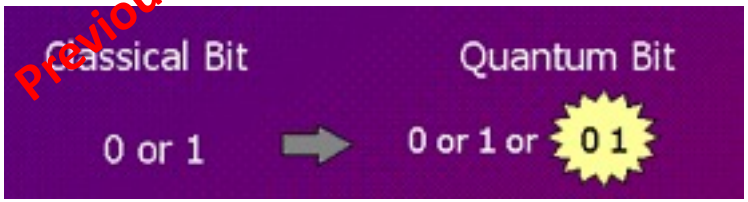
- Currently available for only data transfer.
- Work in progress towards “photonic logic” for designing circuits which use photons:
  - Many challenges



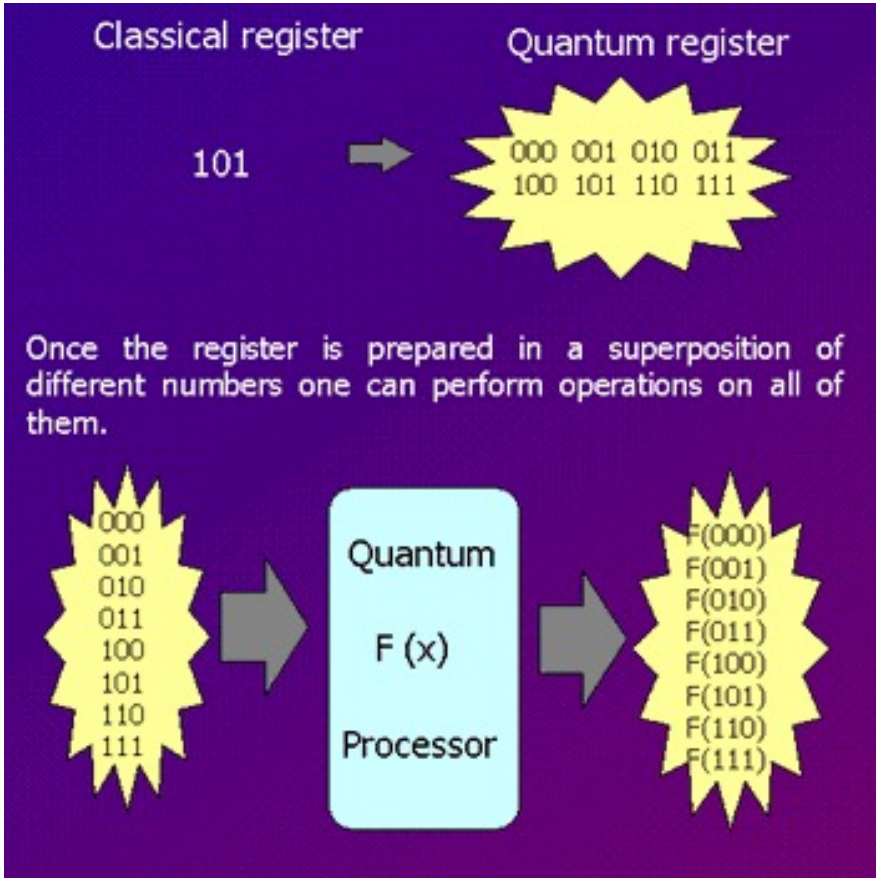
http://en.wikipedia.org/wiki/Optical_computing

New Trend: Quantum Computing

Previously on CENG111!



- There are several problems:
  - Decoherence: The more the qubits, the more their effect on the environment.



http://media.defenseindustrydaily.com/images/PUB_CQC_Cambridge_Quantum_Computing_Explained_lg.png

Read Especially This: http://www.cs.virginia.edu/~robins/The_Limits_of_Quantum_Computers.pdf



Today

- More on Memory
- Peripherals
- Interrupts
- Booting
- Operating System
- Programming



Administrative Notes

- Von Neumann Machine Exam:
  - Preliminary Text has been announced!
  - In-class Exam: 23 October (Friday) – 18:00



MORE ON MEMORY



Memory Types

- Dynamic Memory
  - The voltages stored in capacitors die away with time.
  - Solution?
    - Refresh the memory frequently; i.e., re-write the contents of the memory.
- Static Memory
  - A coupled transistor system stores the information.
  - One of the couples *triggers* the other.
- DRAM is cheaper and more widely used.

http://wiki.xtronics.com/index.php/How_Memory_Works



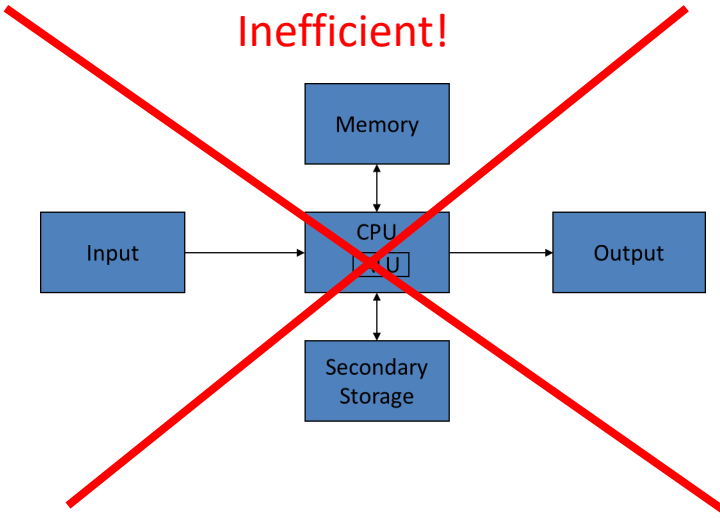
Memory Types

- Volatile Memory:
  - The stored values are lost when power is off.
  - DRAM, SRAM
- Non-volatile:
  - Read-only Memory (ROM), flash memory

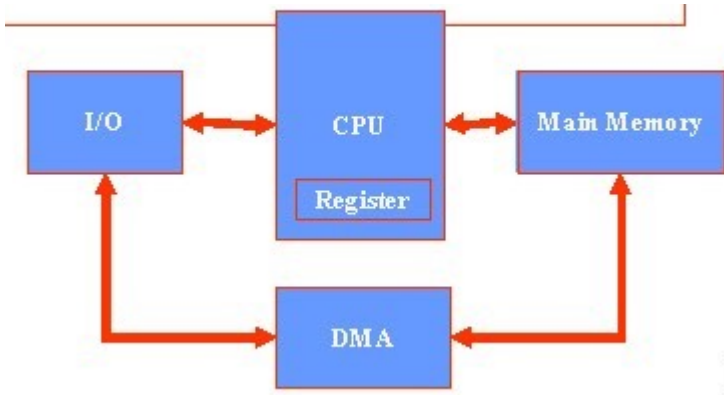
http://wiki.xtronics.com/index.php/How_Memory_Works

Direct Memory Access (DMA)

http://camnetway.blogspot.com/2009/11/interrupt-driven-io.html



- All memory accesses go over the CPU.
  - CPU gets the virtual address,
  - It looks up the physical address,
  - It writes the data to the physical device (memory, in most cases)
- This slows down the CPU for simple and consecutive memory accesses.
- "Modern" computers have a channel for DMA.
- The CPU just initiates the DMA command, and a separate unit (DMA controller) handles the copy of data between a device and a memory.



PERIPHERALS

* You are not responsible from this in the exam.



I/O

- Memory-mapped I/O
  - Treats I/O devices as if they are memory
  - “Load” and “Store” instructions are used for accessing an I/O device
  - Single physical address space for memory and I/O devices
  - Devices: Graphics card, hard drive
- Port-mapped I/O (or isolated I/O)
  - Special instructions for performing I/O
  - Separate bus/pin for I/O devices
  - Devices: Keyboard, mouse, …

Input/Output and Mass Storage

- Communication with outside world and external data storage
  - *Input unit*
    - *"Receiving" section of computer*
    - *Obtains data from input devices*
      - *Usually a keyboard, mouse, disk or scanner*
    - *Places data at disposal of other units*
  - *Output unit*
    - *"Shipping" section of computer*
    - *Puts processed info on various output devices*
      - *Screens, paper printouts, speakers*
    - *Makes info available outside the computer*



Input/Output and Mass Storage

- Human interfaces: monitor, keyboard, mouse
- Archival storage: not dependent on constant power
- External devices vary tremendously from each other

Input/Output and Mass Storage (continued)

- Volatile storage
  - Information disappears when the power is turned off
  - Example: RAM
- Nonvolatile storage
  - Information does not disappear when the power is turned off
  - Example: mass storage devices such as disks and tapes

METU Computer Engineering

Input/Output and Mass Storage (continued)

- Mass *(secondary)* storage devices
  - Direct access storage device
    - Hard drive, CD-ROM, DVD, etc.
    - Uses its own addressing scheme to access data
  - Sequential access storage device
    - Tape drive, etc.
    - Stores data sequentially
    - Used for backup storage these days



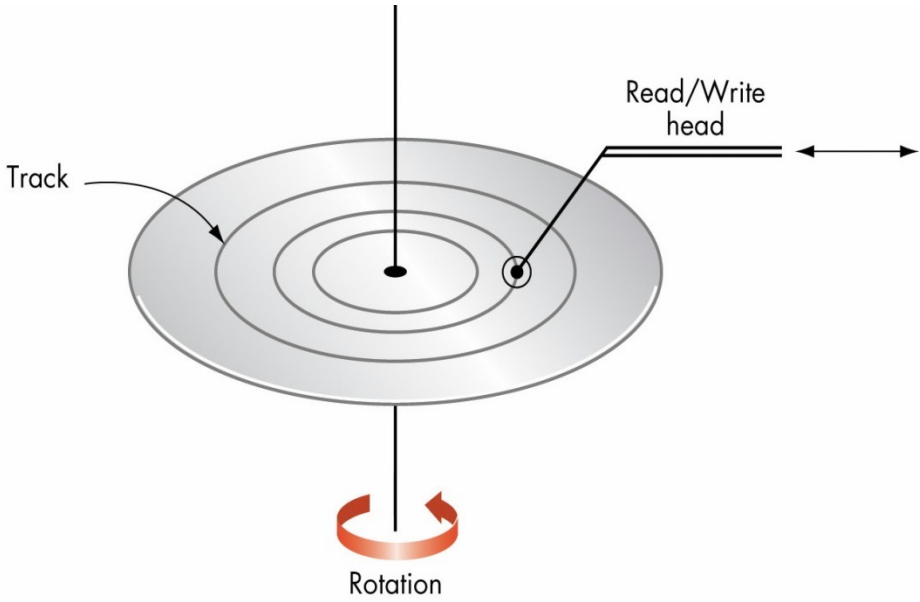
Input/Output and Mass Storage (continued)

- *Mass or Secondary storage unit*
  - Long-term, high-capacity “warehouse”
  - Stores programs or data not currently being used by other units on *secondary storage devices* (like discs)
  - Takes longer to access than primary memory
  - Data location

METU Computer Engineering

Input/Output and Mass Storage (continued)

- Direct access storage devices
- Data stored on a spinning disk
- Disk divided into concentric rings (tracks)
- Read/write head moves from one ring to another while disk spins
- Access time depends on:
  - Time to move head to correct track
  - Time for disc to spin to data location
- We will come back to rotational storage media!



METU Computer Engineering

The IBM 350

- 1956—the first hard disk drive
- 4.4 MB of storage
- Weighed over a ton
- About 1,000 systems built

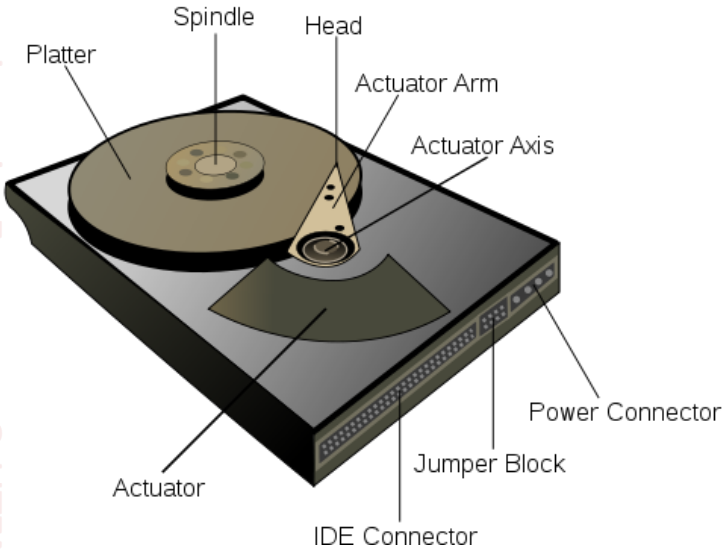


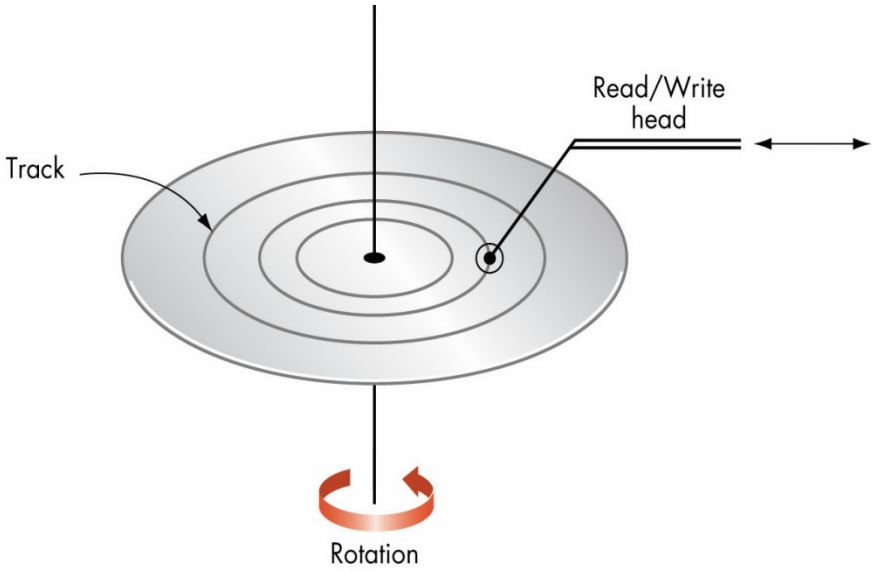
Image from www.ichigo.se/noterat/wp-content;
information from http://en.wikipedia.org/wiki/RAMAC

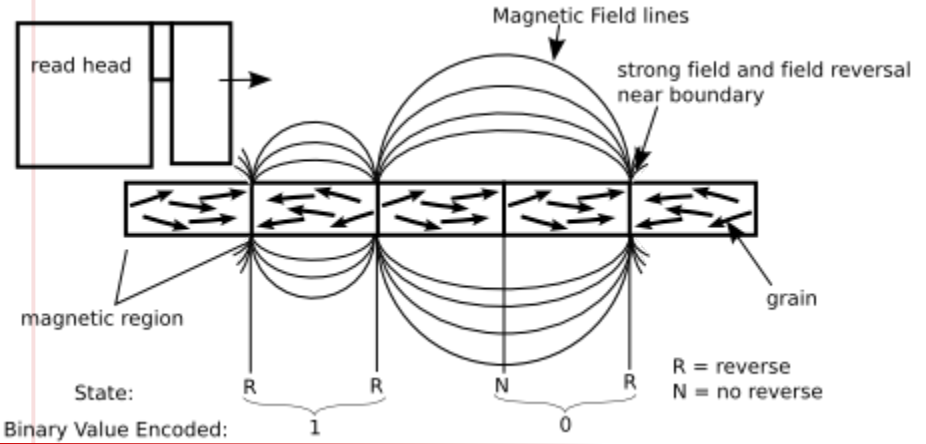
http://dwave.files.wordpress.com/2007/03/20070301_demo_slides.pdf

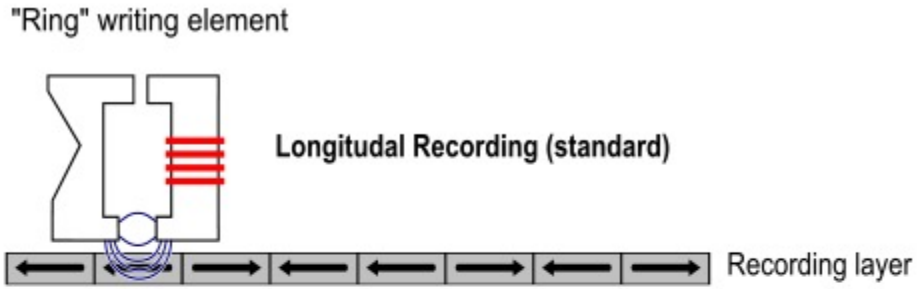
Rotational Media as External Storage

METU Computer Engineering



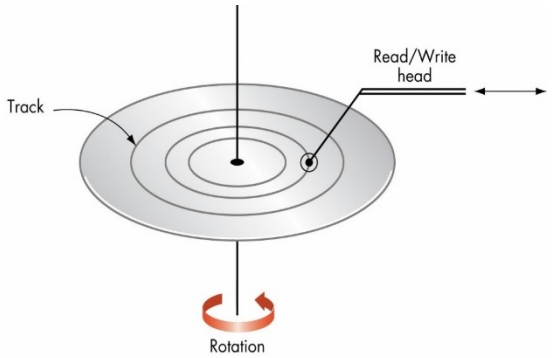






Rotational Drive Performance Characteristics

- For bigger disks, multiple layers & condensed layers are used.
- Access Time = Seek Time + Rotational Latency
  - Seek Time
    - "the time it takes the head to travel to the track of the disk where the data will be read or written"
    - around 9ms
  - Rotational Latency
    - "delay waiting for the rotation of the disk to bring the required disk sector under the read-write head."
    - Depends on Rotations Per Minute (RPM)
- Data Transfer Rate
  - Covers the internal rate & the external rate
  - SATA: 3.0 Gbit/s





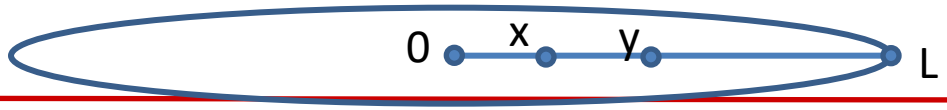
Note that alternative definitions of access time include data transfer rate as well. However, we will stick to this definition.

http://en.wikipedia.org/wiki/Disk_drive_performance_characteristics

One Last Bit of Information Regarding Rotational Media

- Average Seek Time
  - x: track that we want to move the arm to.
  - y: current position of the arm.
  - How can we model it in terms of the number of tracks?
- Average Seek Time = average distance $|x - y|$ multiplied by the time to move the arm by one track.
- The average distance = $L/3$. How?

Average Distance = $$\frac{1}{L^2}\int_0^L\int_0^L |x-y|\,dxdy = L/3$$



Solid State Disk (SSD)

- NAND Flash Memory Chips
- Access through the controller:
  - Faster than hard disk, still slower than RAM
- Better for daily usage
- Not so reliable for offline storage



Transfer of Information
The Concept of Ports

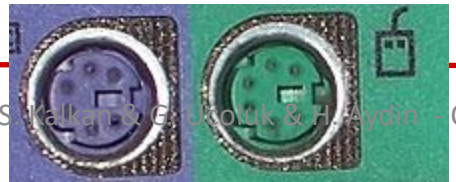
- What is a port, literally?
- What is a computer port, then?
  - SATA, e-SATA, USB, Firewire, Serial, Parallel (e.g., PS/2) ports.











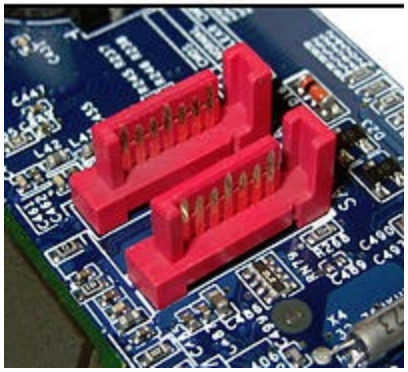




Transfer of Information In/On the Computer

- SATA – Serial Advanced Technology Attachment:
  - Bus interface between motherboard and storage devices, using high-speed cable.
  - Several versions (protocol & encoding is different):
    - SATA1: 1.5Gbit/s
    - SATA2: 3.0Gbit/s
    - SATA3: 6.0Gbit/s
- Alternatives:
  - IDE
  - PATA – Parallel Advanced Technology Attachment





Transfer of Information In/On the Computer

- Firewire – IEEE 1394 Interface
- High-Definition Audio-Video Network Alliance standard interface for audio/video communication.
- Several versions exist.
  - Around 800Mbit/s.
- Alternative:
  - Universal Serial Bus (i.e., USB)





Transfer of Information In/On the Computer

- Universal Serial Bus – USB
- Alternative to Firewire
  - Minor differences exist
- In addition to communication, USB can provide power to the device.
- Several versions exist: 1.0, 2.0, 3.0 and 4.0.
  - 2.0 is the current widely used standard.
  - 3.0 is available ~~but still not widely used~~.
  - 4.0 is becoming popular with the USB-C connector.



Transfer of Information Between Computers

- Ethernet
  - For inter-net as well as intra-net.
- Different versions:
  - 100Mbit/s
  - 1Gbit/s
- Ones that are not widely supported yet:
  - 10Gbit/s
  - 100Gbit/s





METU Computer Engineering



INTERRUPTS

* You are not responsible from this in the exam.

“Are we there yet?”



https://youtu.be/basofea2UEs?t=24



Interrupts

- When you type a character at the keyboard, the keyboard controller transmits the character to the CPU.
- However, the CPU is busy all the time and the keyboard controller has a limited storage; usually, only for one character.
- How to get the CPU's attention?
- Polling vs interrupt.
  - Polling: a phone without a bell.

http://www.atarimagazines.com/compute/issue149/60_Interrupts_made_easy.php



Interrupts

- I/O, time management, power-off signals, traps, …
- When an interrupt is received,
  - CPU stops the current task/program.
  - It saves the current “context”. How?
  - It executes the code necessary for the received interrupt.



Interrupt Table

- After booting, devices and operating systems register service routines that tell the computer "what to do" on different interrupts.
- The addresses of the routines that will handle the interrupts are stored in the interrupt table.

Interrupt vector	Description
00h	CPU: Executed after an attempt to divide by zero or when the quotient does not fit in the destination
01h	CPU: Executed after every instruction while the trace flag is set
02h	CPU: NMI, used e.g. by POST for memory errors
03h	CPU: The lowest non-reserved interrupt, it is used exclusively for debugging, and the INT 03 handler is always implemented by a debugging program
04h	CPU: Numeric Overflow. Usually caused by the INTO instruction when the overflow flag is set.
05h	Executed when Shift-Print screen is pressed, as well as when the BOUND instruction detects a bound failure.
06h	CPU: Called when the Undefined Opcode (invalid instruction) exception occurs. Usually installed by the operating system.
07h	CPU: Called when an attempt was made to execute a floating-point instruction and no numeric coprocessor was available.
08h	**IRQ0: Implemented by the system timing component; called 18.2 times per second (once every 55 ms) by the PIC**
09h	**IRQ1: Called after every key press and release (as well as during the time when a key is being held)**
0Bh	**IRQ3: Called by serial ports 2 and 4 (COM2/4) when in need of attention**
0Ch	**IRQ4: Called by serial ports 1 and 3 (COM1/3) when in need of attention**
0Dh	**IRQ5: Called by hard disk controller (PC/XT) or 2nd parallel port LPT2 (AT) when in need of attention**
0Eh	**IRQ6: Called by floppy disk controller when in need of attention**
0Fh	**IRQ7: Called by 1st parallel port LPT1 (printer) when in need of attention**

Timer Interrupt

- A system timer generates interrupts on every “tic”
- Called 18.2 times every second; once every 55ms.
  - Programmable
- Why is it needed?
  - Memory refreshing
  - OS time management
    - OS reads the initial time when booted
    - It increments the initial value with every time interrupt



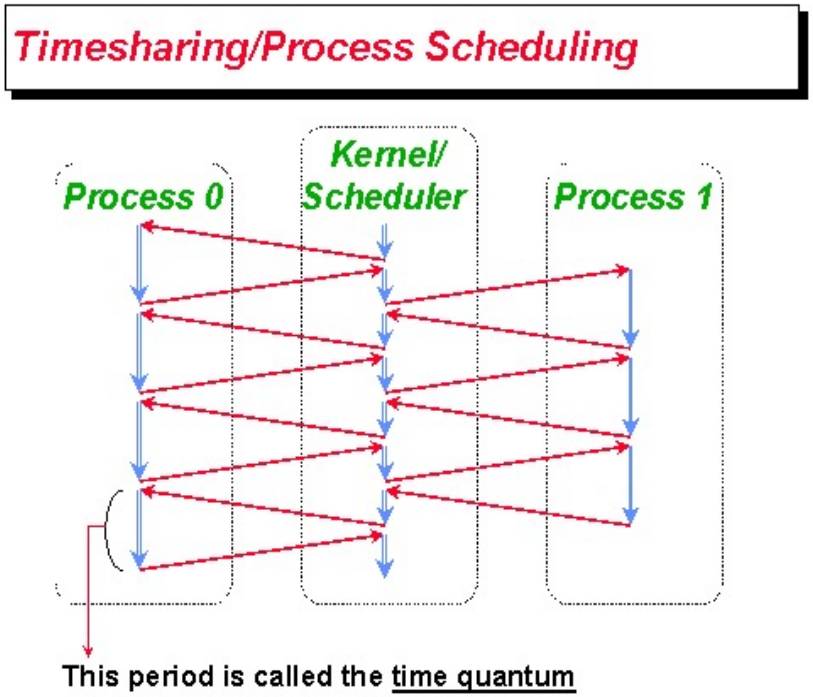
Masking Interrupts

- Disabling interrupts for a limited time.
- When is it needed?
  - Busy, handling an important interrupt already
  - Being interrupted by the interrupt being handled.

http://en.wikipedia.org/wiki/Interrupt_mask_register

Interrupts & Multi-tasking

- Most of the time, there is more than one program/process running.
- Each program runs for a fixed amount of time T.
  - T depends on the priority of the program/process.
- After T is finished, the CPU “interrupts”.



http://6004.csail.mit.edu/Spring98/Lectures/lect21/sld009.htm





What happens after that power button

BOOTING YOUR COMPUTER

* You are not responsible from this in the exam.

For more details, see e.g.: https://neosmart.net/wiki/mbr-boot-process/



Booting the Computer

- Basic Input-Output System (BIOS)
  - BIOS is on a Read-Only Memory (ROM)
  - When starting, CPU addresses the address space of the ROM and starts executing the BIOS.
  - BIOS first initiates self-check of hardware (Power-on self test – POST)
  - BIOS loads the rest of the BIOS into the RAM (because ROM is slow)
  - Then, it goes over non-volatile storage devices to find something bootable.
- Boot Device & Master Boot Record (MBR)
  - BIOS loads MBR from the boot device
  - MBR is not OS specific
  - MBR checks for a partition to boot
  - Loads OS (starts OS kernel)



Master Boot Record

- First 512-bytes of a partitioned data storage device; e.g., a disk.
- Responsibilities:
  - Holding the partition table
  - Helps booting of operating systems
  - Stores the ID of the disk (not used)
- A part of these 512-bytes is a code that searches the "primary" partitions and finds the one that is marked "active" in the table. The "volume boot record" of the active partition boots the OS.

Master Boot Record

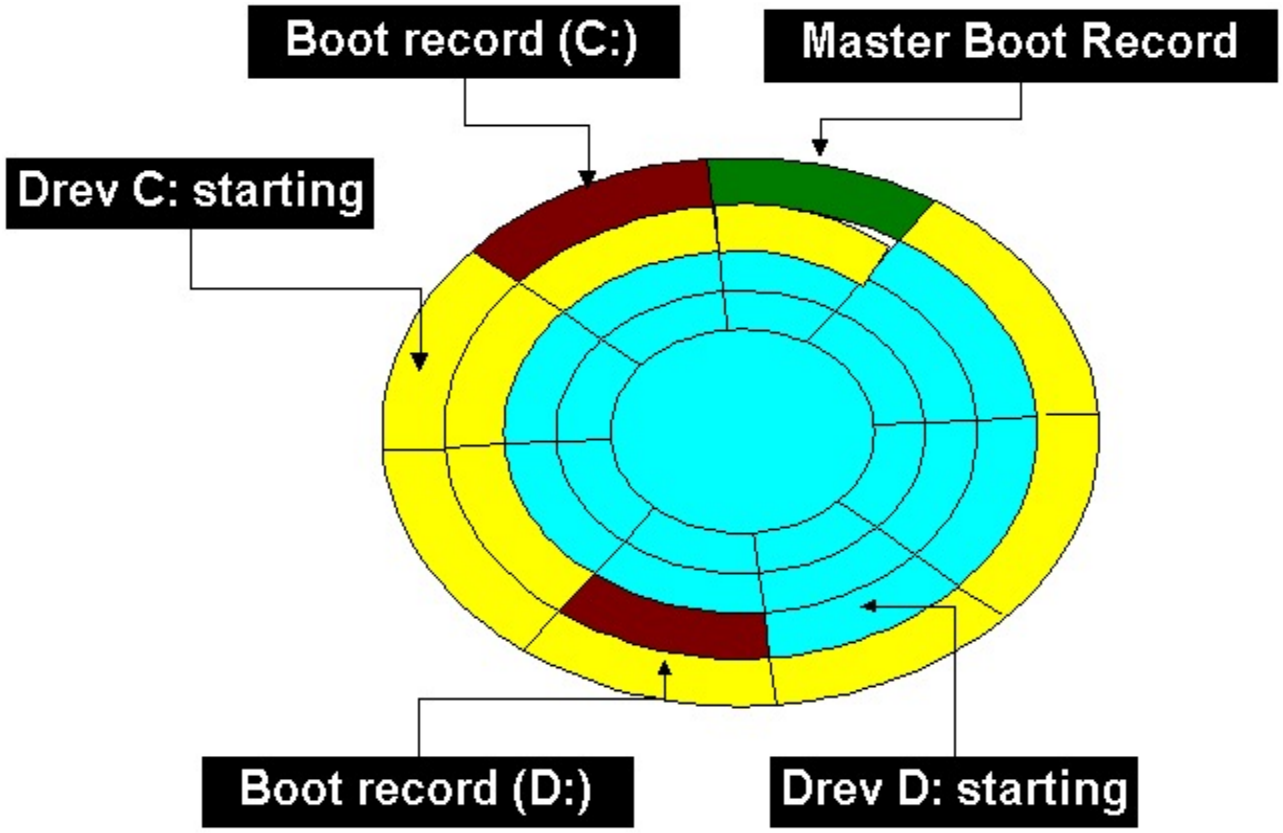


Fig: https://techstroke.com/

Master Boot Record

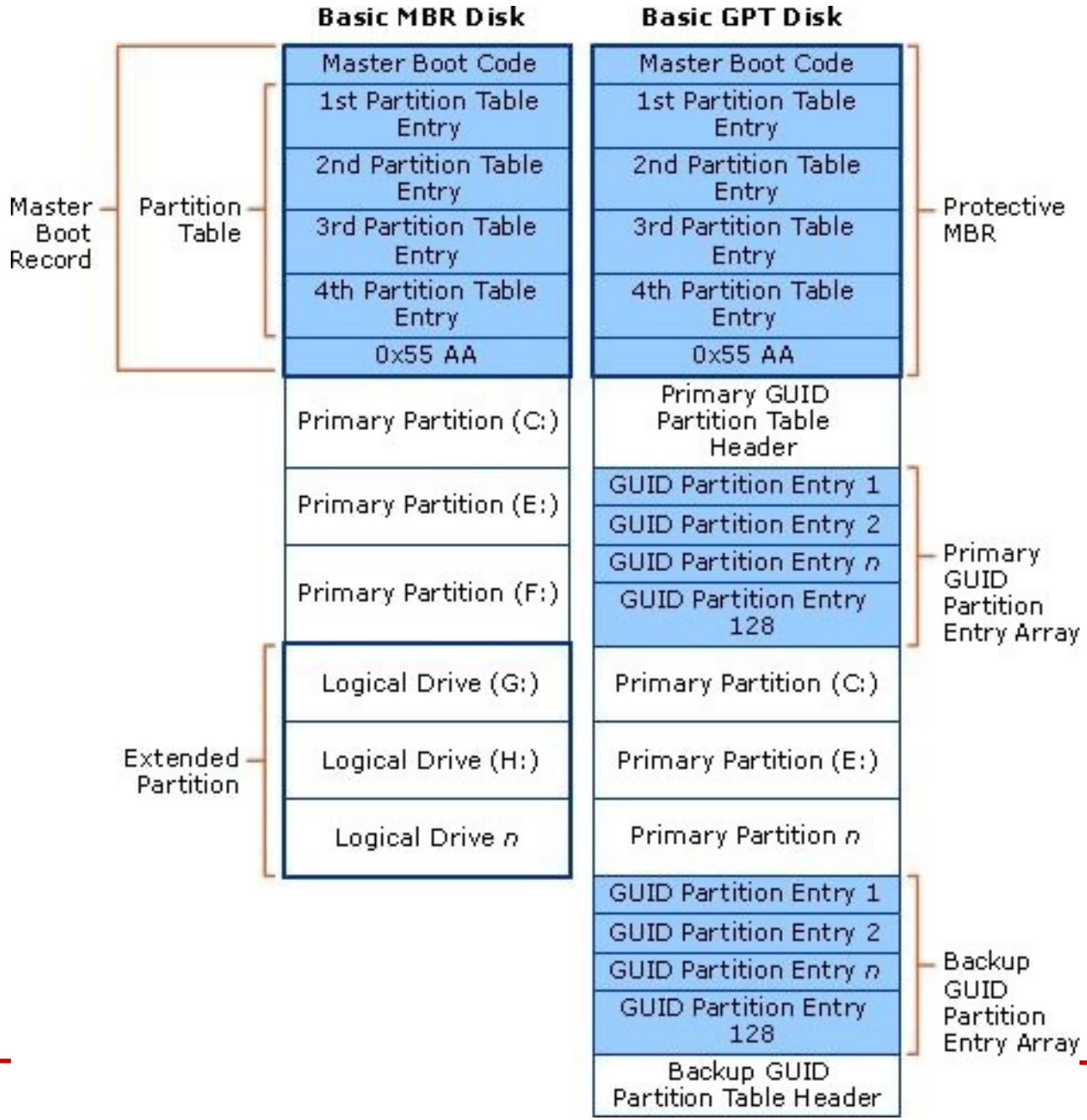
Structure of a Master Boot Record

Address			Description		Size in bytes
Hex	Oct	Dec			
0000	0000	0	code area		440 (max. 446)
01B8	0670	440	disk signature (optional)		4
01BC	0674	444	Usually nulls; 0x0000		2
01BE	0676	446	Table of primary partitions (Four 16-byte entries, IBM partition table scheme)		64
01FE	0776	510	55h	MBR signature; 0xAA55[1]	2
01FF	0777	511	AAh		
MBR, total size: 446 + 64 + 2 =					512

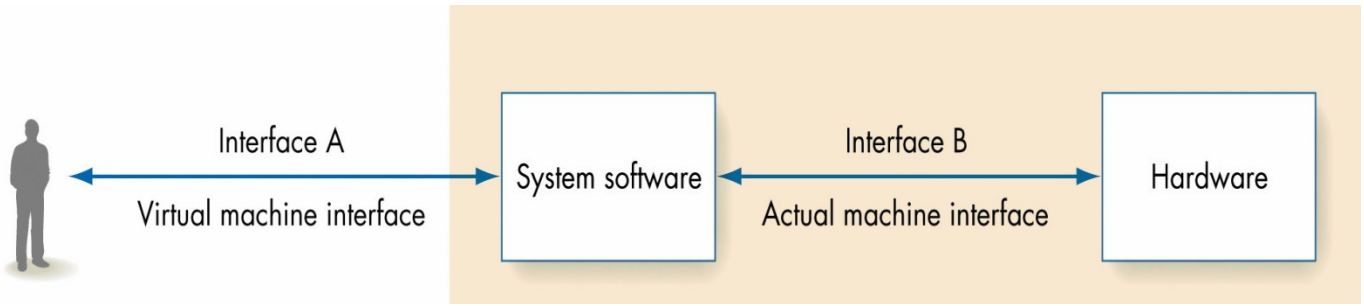


UEFI & GPT

- Unified Extensible Firmware Interface
  - Extends BIOS
  - managed by a group of chipset, hardware, system, firmware, and operating system vendors called the UEFI Forum
  - Faster hardware check compared to BIOS
  - Allows developers to add applications, enabling UEFI to be a lightweight OS
  - Provides secure boot, disables unauthorized applications from booting
  - Provides GUI
- GPT (GUID Partition Table) – required by UEFI
  - Extends MBR
  - GUID: Globally (Universally) unique identifier. Easily creatable almost unique identifier for partitions
  - Supports bigger disks (> 2TB), more partitions (up to 128)
  - It has backup at the end, which can help recover partitions



https://www.dell.com/community/Latitude/BIOS-setting-s-to-Install-Windows-Using-UEFI/td-p/7258597/page/3



OPERATING SYSTEM

* You are not responsible from this in the exam.



Introduction

- Von Neumann computer
  - “Naked machine”
  - Hardware without any helpful user-oriented features
  - Extremely difficult for a human to work with
- An interface between the user and the hardware is needed to make a Von Neumann computer usable



Introduction (continued)

- Tasks of the interface
  - Hide details of the underlying hardware from the user
  - Present information in a way that does not require in-depth knowledge of the internal structure of the system



Introduction (continued)

- Tasks of the interface (continued)
  - Allow easy user access to the available resources
  - Prevent accidental or intentional damage to hardware, programs, and data



System Software: The Virtual Machine

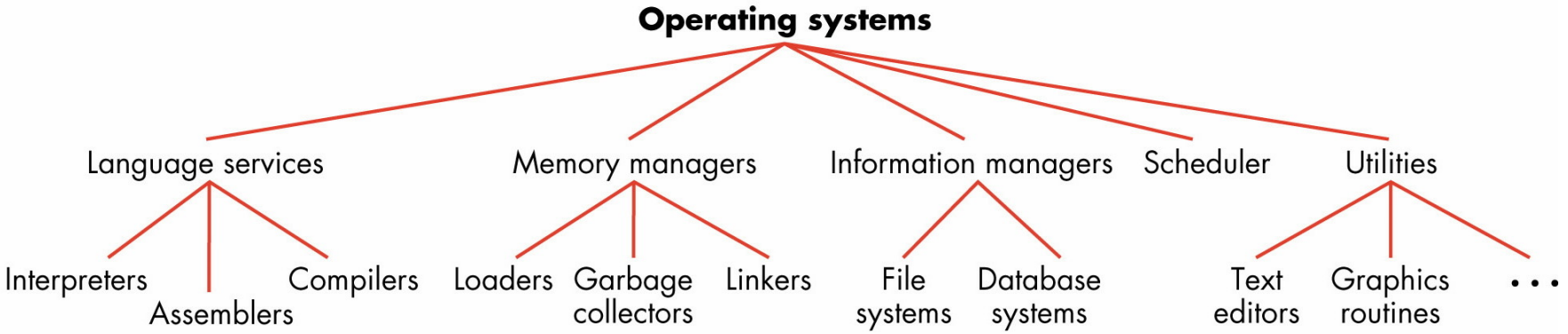
METU Computer Engineering

- System software
  - Acts as an intermediary between users and hardware
  - Creates a virtual environment for the user that hides the actual computer architecture
- Virtual machine (or virtual environment)
  - Set of services and resources created by the system software and seen by the user



Types of System Software

- System software is a collection of many different programs
- Operating system
  - Controls the overall operation of the computer
  - Communicates with the user
  - Determines what the user wants
  - Activates system programs, application packages, or user programs to carry out user requests



Responsibilities of an OS



Types of System Software (continued)

METU Computer Engineering

- User interface
  - Graphical user interface (GUI) provides graphical control of the capabilities and services of the computer
- Language services
  - Assemblers, compilers, and interpreters
  - Allow you to write programs in a high-level, user-oriented language, and then execute them



Types of System Software (continued)

METU Computer Engineering

- Memory managers
  - Allocate and retrieve memory space
- Information managers
  - Handle the organization, storage, and retrieval of information on mass storage devices
- I/O systems
  - Allow the use of different types of input and output devices

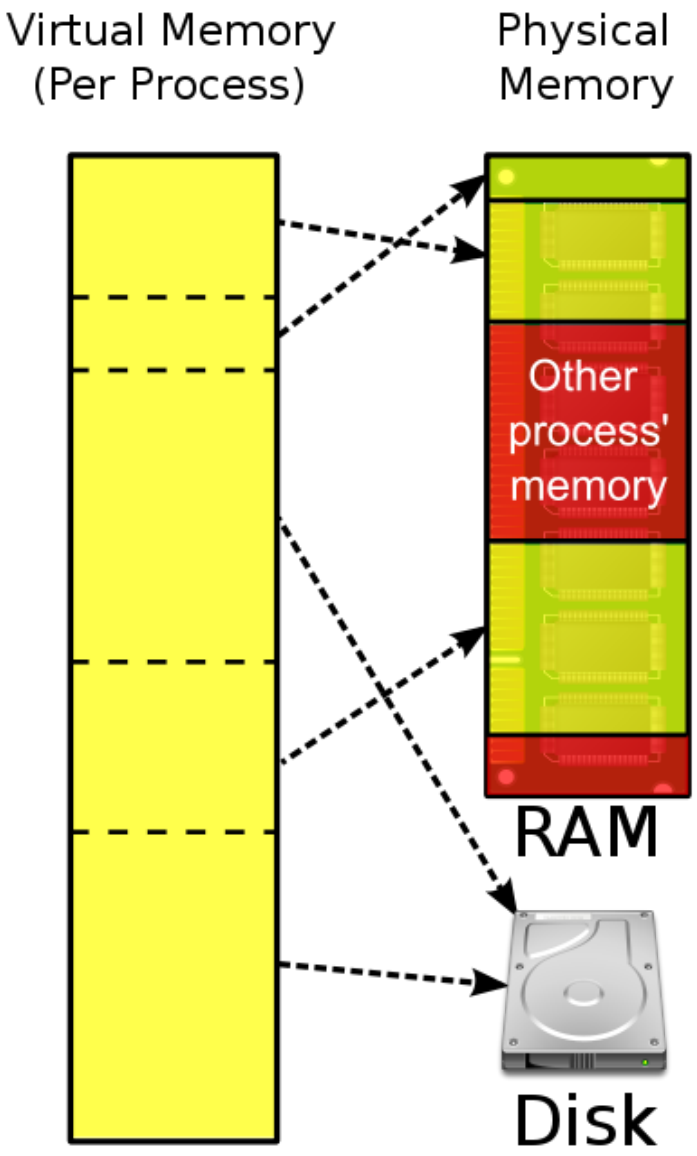


Types of System Software (continued)

- Scheduler
  - Keeps a list of programs ready to run and selects the one that will execute next
- Utilities
  - Collections of library routines that provide services either to user or other system routines

Virtual Memory

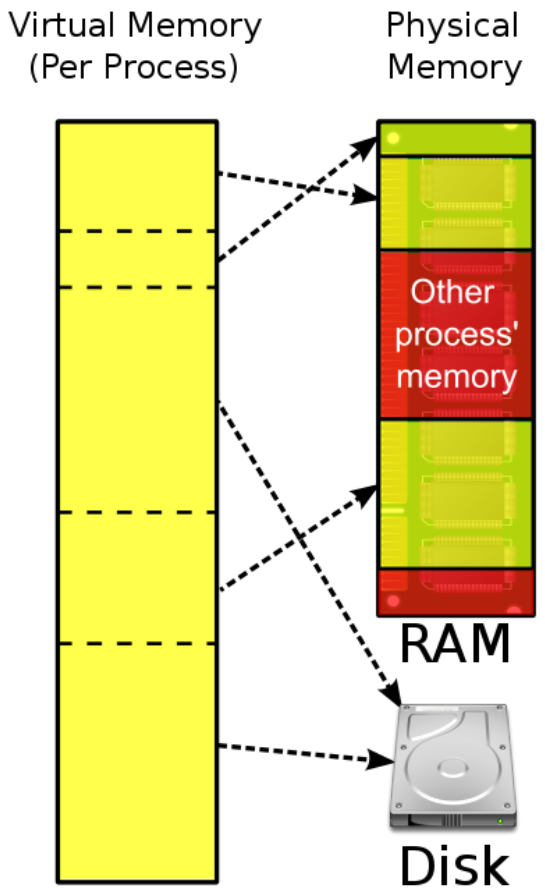
- Memory management technique for “abstraction” over several RAM and disk storage devices.
- A program sees only “one RAM”; it does not care about the details.
- Not suitable for real-time systems.
- “Page”s are used in Virtual Memory.



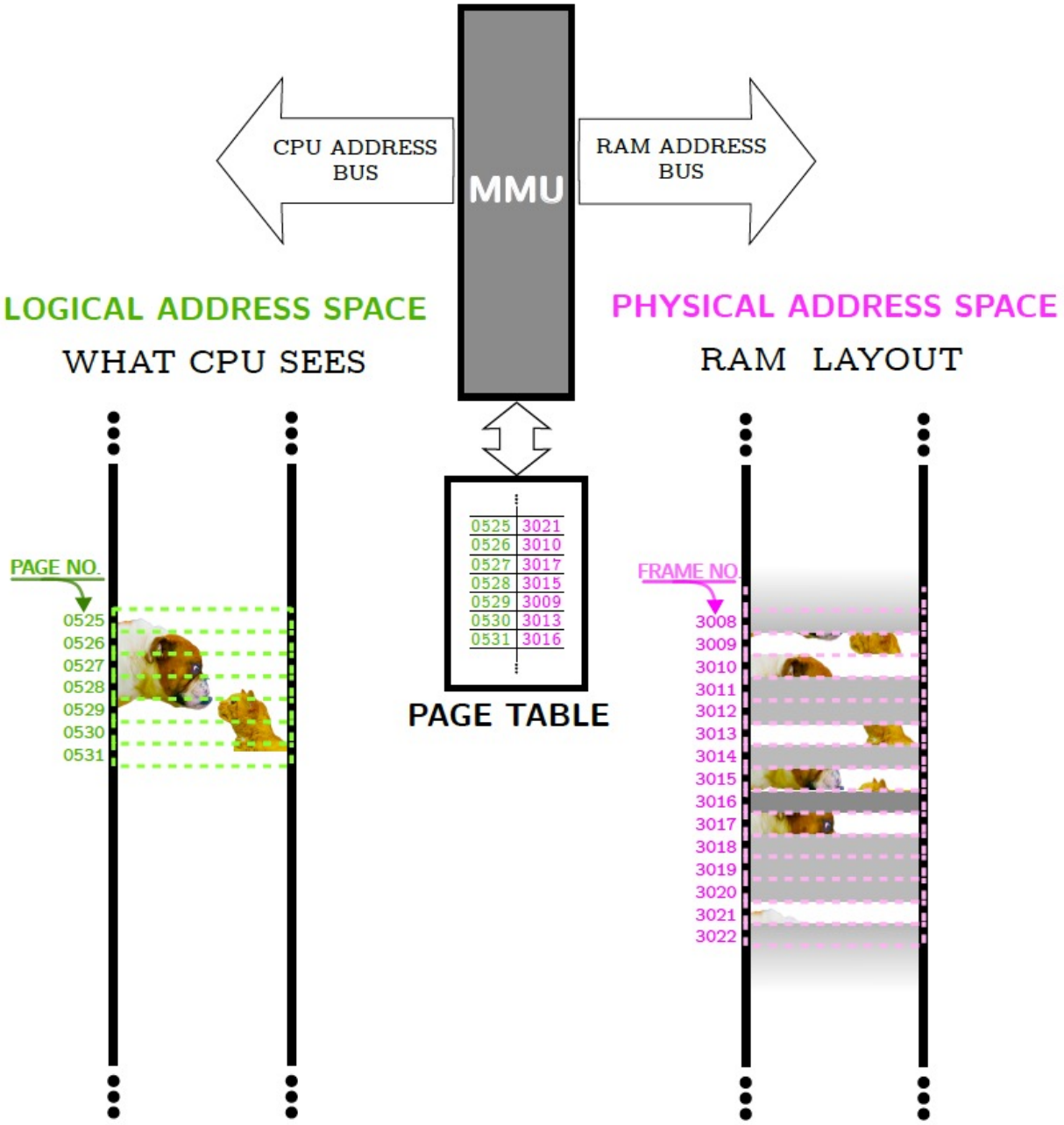
http://en.wikipedia.org/wiki/Virtual_memory

Virtual Memory
- Paging -

- A **page**: at least 4KB of consecutive virtual memory addresses.
- An application program is stored in a set of pages.
- A **page table** maps the logical/virtual addresses of the pages with the physical addresses.



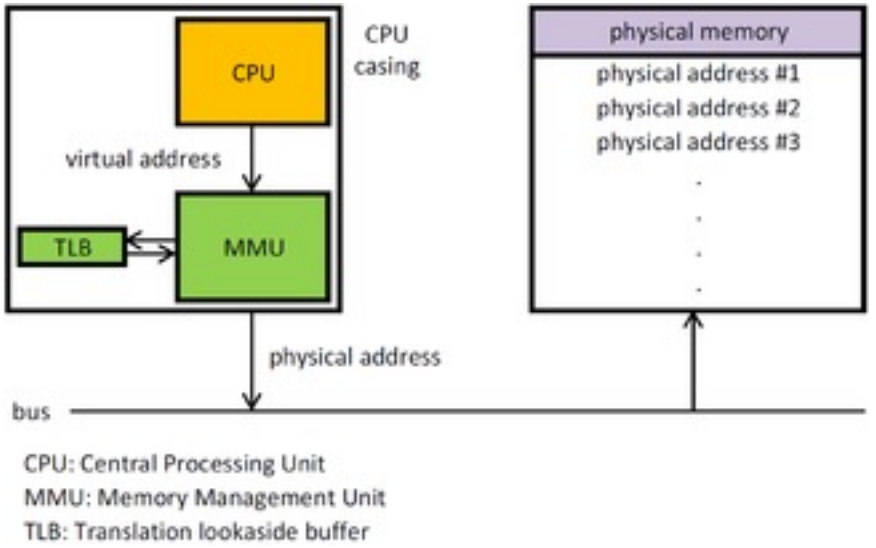
http://en.wikipedia.org/wiki/Virtual_memory


CPU ADDRESS BUS
MMU
RAM ADDRESS BUS
LOGICAL ADDRESS SPACE
WHAT CPU SEES
PHYSICAL ADDRESS SPACE
RAM LAYOUT
PAGE NO.
0525
0526
0527
0528
0529
0530
0531
0525 3021
0526 3010
0527 3017
0528 3015
0529 3009
0530 3013
0531 3016
PAGE TABLE
FRAME NO
3008
3009
3010
3011
3012
3013
3014
3015
3016
3017
3018
3019
3020
3021
3022

Memory Management Unit (MMU)

METU Computer Engineering

- Stores a lookup table in memory or a small device called Translation Lookaside Buffer (TLB)
- Can keep track of whether a page is on RAM, in use, last-used, updated, ..
- Non-contiguous physical addresses can be mapped to contiguous virtual addresses



https://en.wikipedia.org/wiki/Memory_management_unit



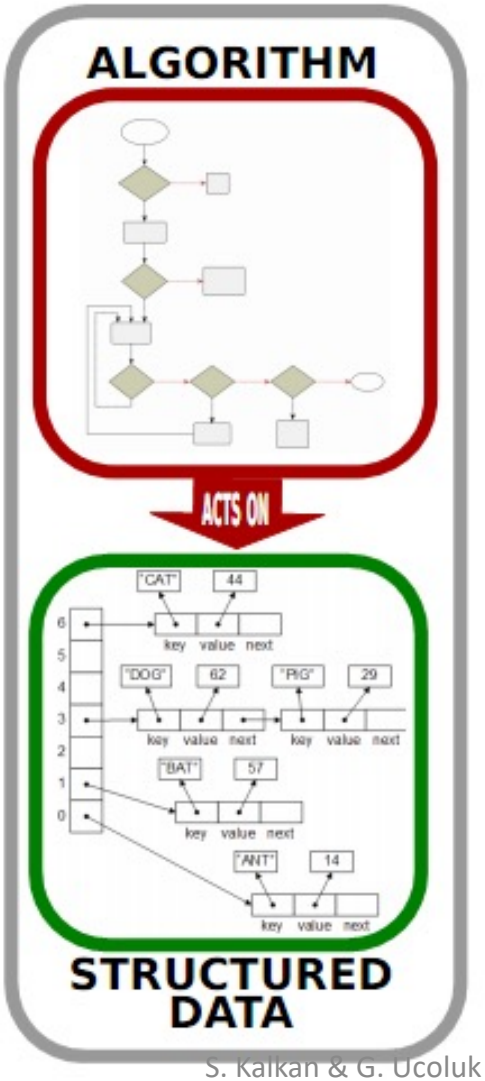
PROGRAMMING

Program, Programming

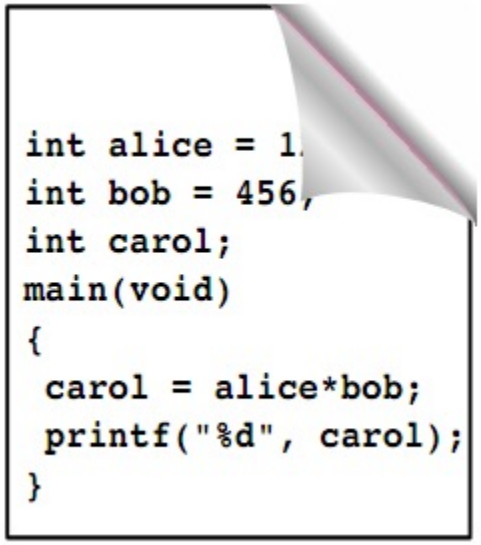


WORLD PROBLEM

TRANSFORMED



IMPLEMENTED



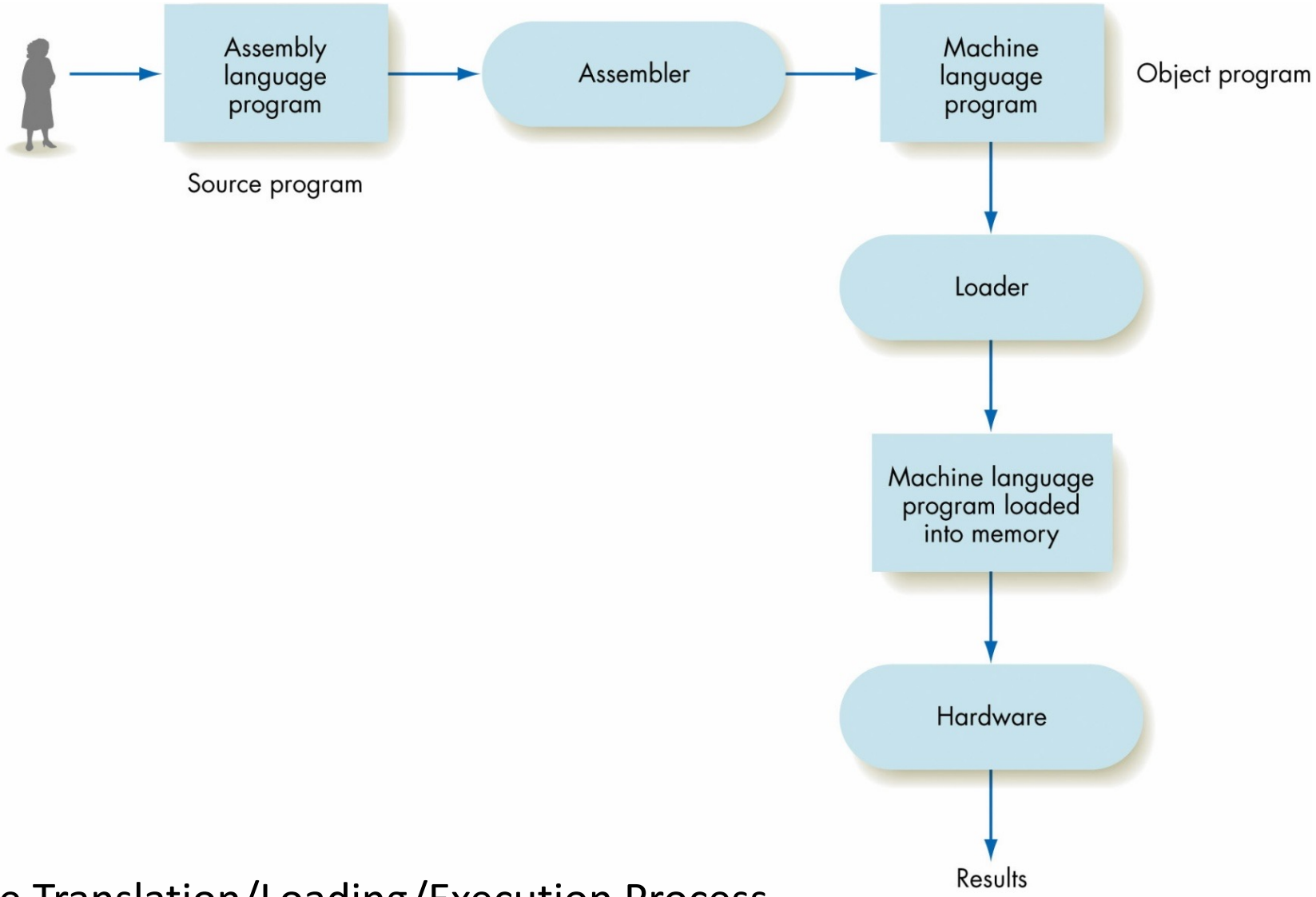
```
int alice = 1
int bob = 456,
int carol;
main(void)
{
 carol = alice*bob;
 printf("%d", carol);
}
```

PROGRAM



Program, Programming

- Program:
  - “a series of steps to be carried out or goals to be accomplished”
- A recipe for cooking a certain dish is also a program (but not a computer program).



The Translation/Loading/Execution Process



Assembly Language (continued)

- Source program
  - An assembly language program
- Object program
  - A machine language program
- Assembler
  - Translates a source program into a corresponding object program



Assembly Language (continued)

- Advantages of writing in assembly language rather than machine language
  - Use of symbolic operation codes rather than numeric (binary) ones
  - Use of symbolic memory addresses rather than numeric (binary) ones
  - Pseudo-operations that provide useful user-oriented services such as data generation

```
.BEGIN      --This must be the first line of the program.
    ⋮       --Assembly language instructions like those in Figure 6.5.
   HALT     --This instruction terminates execution of the program
    ⋮       --Data generation pseudo-ops such as
            --.DATA are placed here, after the HALT.
.END  --This must be the last line of the program.
```

Structure of a Typical Assembly Language Program



Examples of Assembly Language Code

METU Computer Engineering

- Arithmetic expression

  A = B + C – 7

  (Assume that B and C have already been assigned values)

Examples of Assembly Language Code (continued)

METU Computer Engineering

- Assembly language translation

```
            LOAD        B       --Put the value B into register R.
            ADD         C       --R now holds the sum (B + C).
            SUBTRACT SEVEN      --R now holds the expression (B + C - 7).
            STORE       A       --Store the result into A.
              :
              :                 --These data should be placed after the HALT.
A:          .DATA       0
B:          .DATA       0
C:          .DATA       0
SEVEN:      .DATA       7       --The constant 7.
```



Examples of Assembly Language Code (continued)

METU Computer Engineering

- Problem
  - Read in a sequence of non-negative numbers, one number at a time, and compute their sum
  - When you encounter a negative number, print out the sum of the non-negative values and stop

STEP	OPERATION
1	Set the value of Sum to 0
2	Input the first number *N*
3	While *N* is not negative do
4	Add the value of *N* to Sum
5	Input the next data value *N*
6	End of the loop
7	Print out Sum
8	Stop

Algorithm to Compute the Sum of Numbers

METU Computer Engineering

```
            .BEGIN                  --This marks the start of the program.
            CLEAR       SUM         --Set the running sum to 0 (line 1).
            IN          N           --Input the first number N (line 2).
--The next three instructions test whether N is a negative number (line 3).
AGAIN:      LOAD        ZERO        --Put 0 into register R.
            COMPARE     N           --Compare N and 0.
            JUMPLT      NEG         --Go to NEG if N < 0.
--We get here if N ≥ 0. We add N to the running sum (line 4).
            LOAD        SUM         --Put SUM into R.
            ADD         N           --Add N. R now holds (N + SUM).
            STORE       SUM         --Put the result back into SUM.
--Get the next input value (line 5).
            IN          N
--Now go back and repeat the loop (line 6).
            JUMP        AGAIN
--We get to this section of the program only when we encounter a negative value.
NEG:        OUT         SUM         --Print the sum (line 7)
            HALT                    --and stop (line 8).
--Here are the data generation pseudo-ops
SUM:        .DATA       0           --The running sum goes here.
N:          .DATA       0           --The input data are placed here.
ZERO:       .DATA       0           --The constant 0.
--Now we mark the end of the entire program.
            .END
```

Assembly Language Program to Compute the Sum of Nonnegative Numbers



METU Computer Engineering

Translation and Loading

- Before a source program can be run, an assembler and a loader must be invoked
- Assembler
  - Translates a symbolic assembly language program into machine language
- Loader
  - Reads instructions from the object file and stores them into memory for execution



Translation and Loading (continued)

- Assembler tasks
  - Convert symbolic op codes to binary
  - Convert symbolic addresses to binary
  - Perform assembler services requested by the pseudo-ops
  - Put translated instructions into a file for future use

Programming Languages

C code for the example problem:

```
int alice = 123;
int bob = 456;
int carol;
main(void)
{
   carol = alice*bob;
}
```



Programming Languages

METU Computer Engineering

Assembly code for the example problem:

```
main:
        pushq   %rbp
        movq    %rsp, %rbp
        movl    alice(%rip), %edx
        movl    bob(%rip), %eax
        imull   %edx, %eax
        movl    %eax, carol(%rip)
        movl    $0, %eax
        leave
        ret
alice:
        .long   123
bob:
        .long   456
```

Programming Languages

Machine code for a simple problem/program:

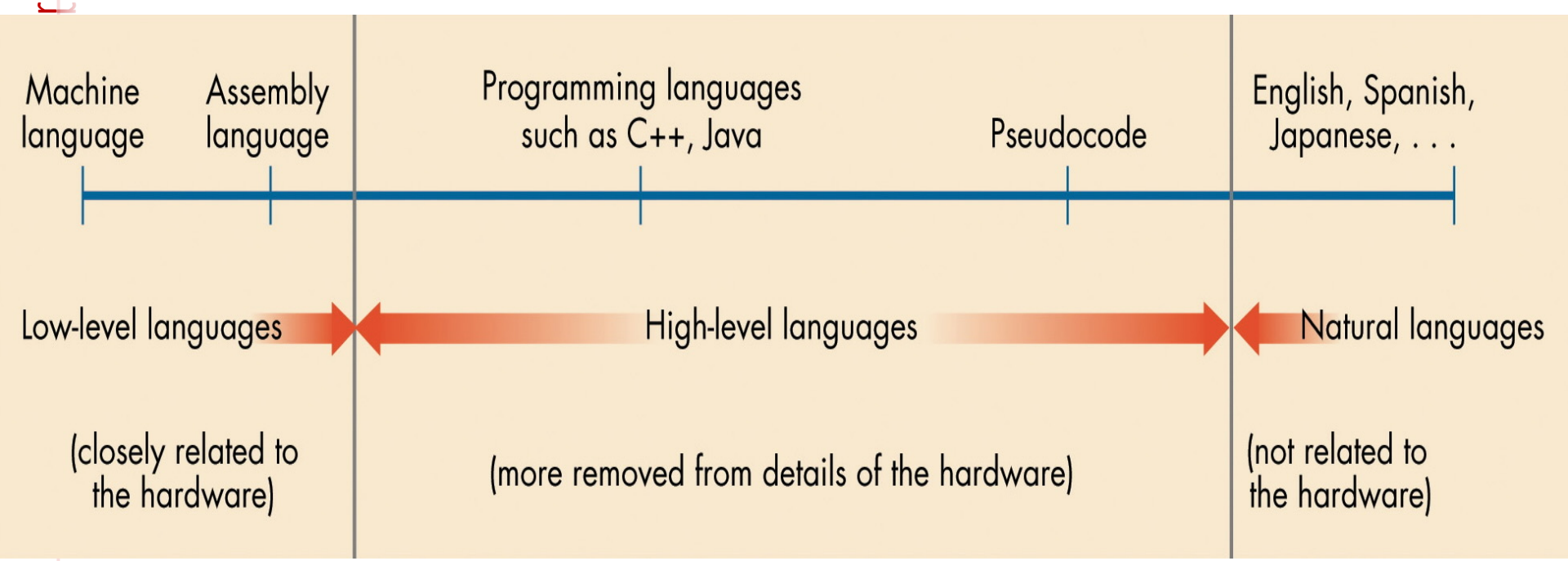
```
01010101 01001000 10001001 11100101 10001011 00010101 10110010 00000011
00100000 00000000 10001011 00000101 10110000 00000011 00100000 00000000
00001111 10101111 11000010 10001001 00000101 10111011 00000011 00100000
00000000 10111000 00000000 00000000 00000000 00000000 11001001 11000011
...
11001000 00000001 00000000 00000000  00000000 00000000
```

```
01010101 01001000 10001001 11100101 10001011 00010101 10110010 00000011
00100000 00000000 10001011 00000101 10110000 00000011 00100000 00000000
00001111 10101111 11000010 10001001 00000101 10111011 00000011 00100000
00000000 10111000 00000000 00000000 00000000 00000000 11001001 11000011
...
11001000 00000001 00000000 00000000  00000000 00000000
```

```
main:
        pushq   %rbp
        movq    %rsp, %rbp
        movl    alice(%rip), %edx
        movl    bob(%rip), %eax
        imull   %edx, %eax
        movl    %eax, carol(%rip)
        movl    $0, %eax
        leave
        ret
alice:
        .long   123
bob:
        .long   456
```

```
int alice = 123;
int bob = 456;
int carol;
main(void)
{
  carol = alice*bob;
}
```

The Spectrum of Programming Languages



- There is a limit to how high a language can get.
- Why can't we write programs in our spoken language?



Zoo of Programming Languages

- Around 700 programming languages!

- But, why do we not have a programming language that serves all paradigms/purposes/requirements?

Choosing a PL

- Ex: Moving soil with a shovel and a grader





Factors that affect choosing a PL

- Domain & Technical Nature of the Problem

a) Finding the pixels of an image with RGB value of [202,130,180] with a tolerance of 5.4% in intensity.

b) A proof system for planar geometry problems.

c) A computer game platform which will be used as a whole or in parts and may get extended even rewritten by programmers at various levels.

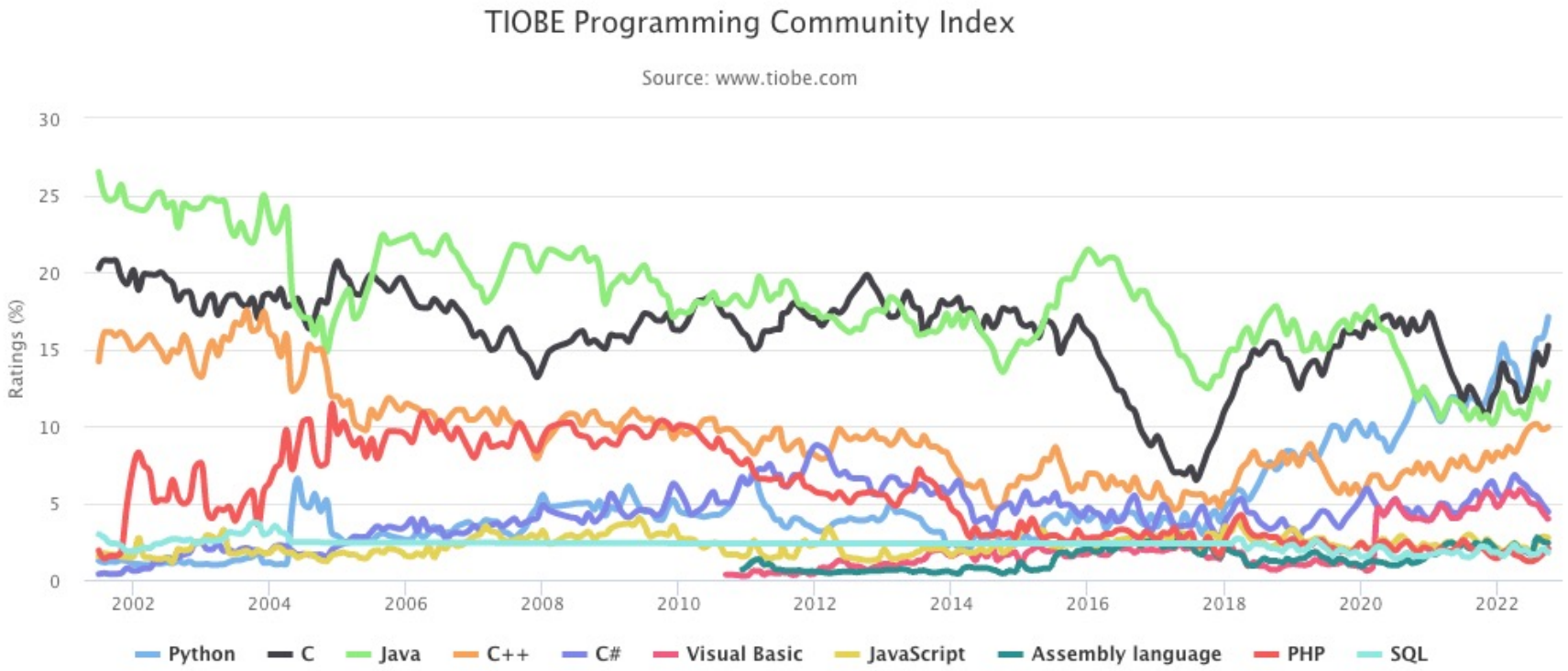
d) Payroll printing.



Factors that affect choosing a PL

- Personal taste and preference
- Circumstance-imposed constraints
  - e.g., time limit.
- Current trend

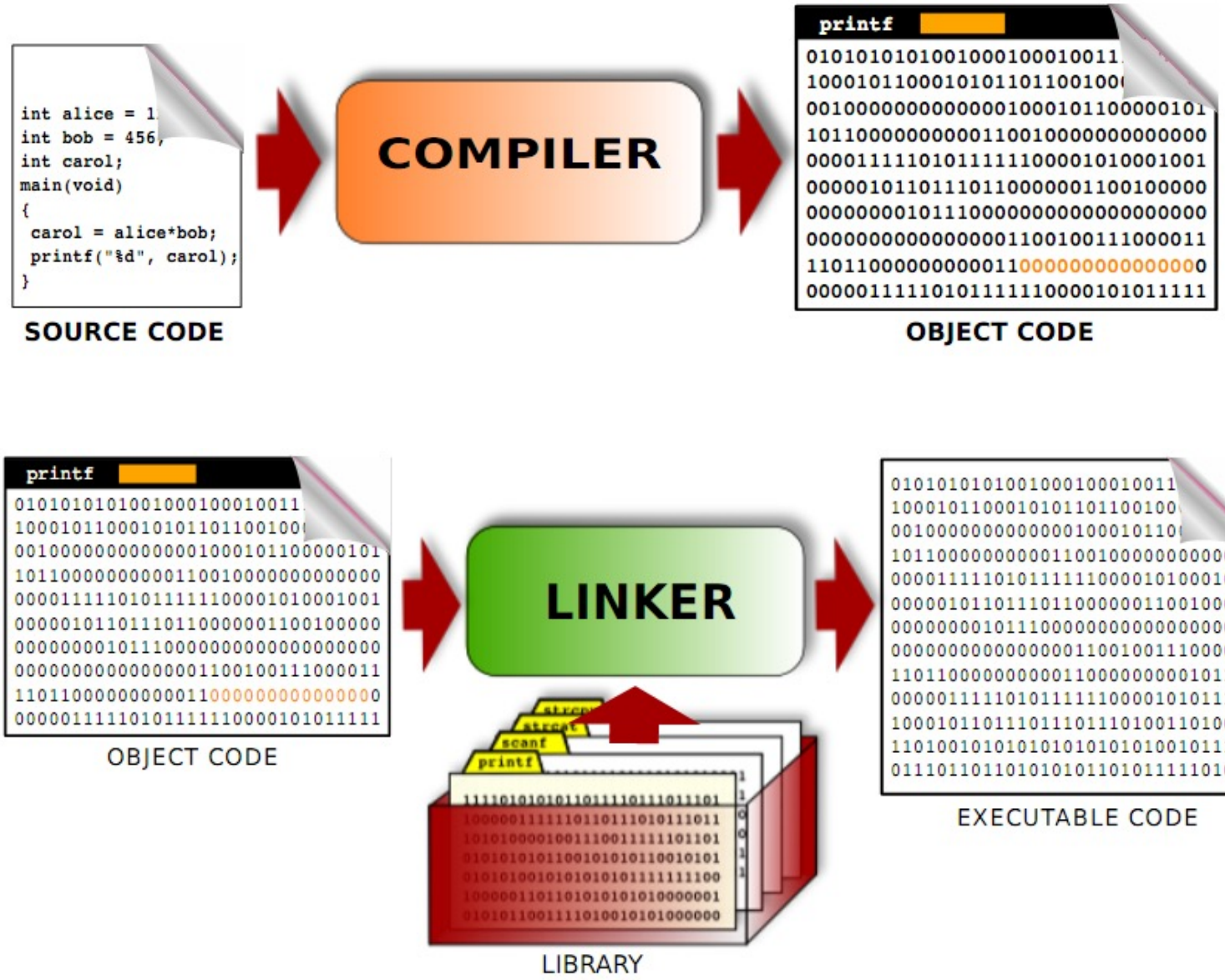
Current trend



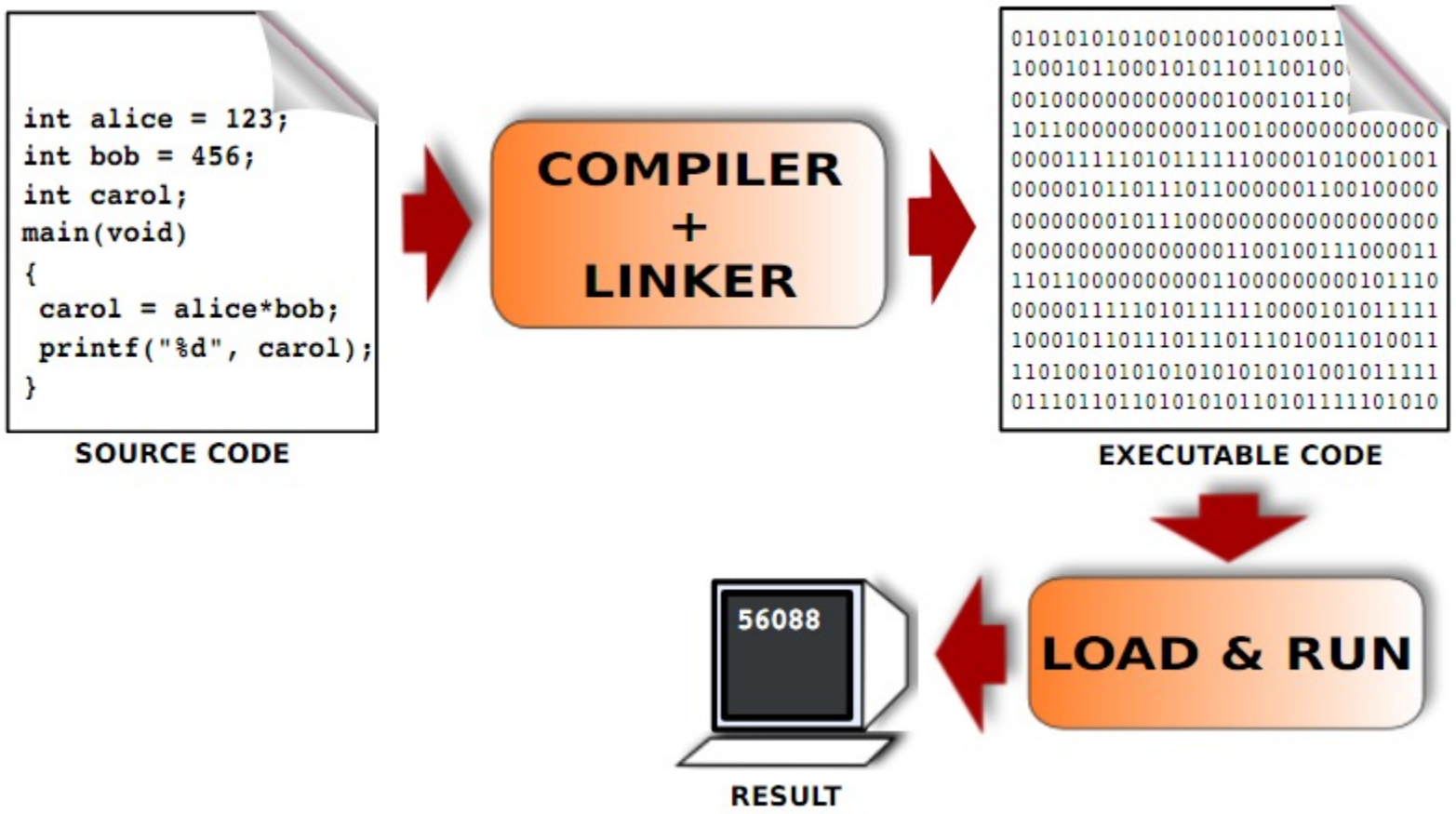
http://www.tiobe.com/index.php/content/paperinfo/tpci/index.html

How are languages implemented

COMPILATIVE APPROACH



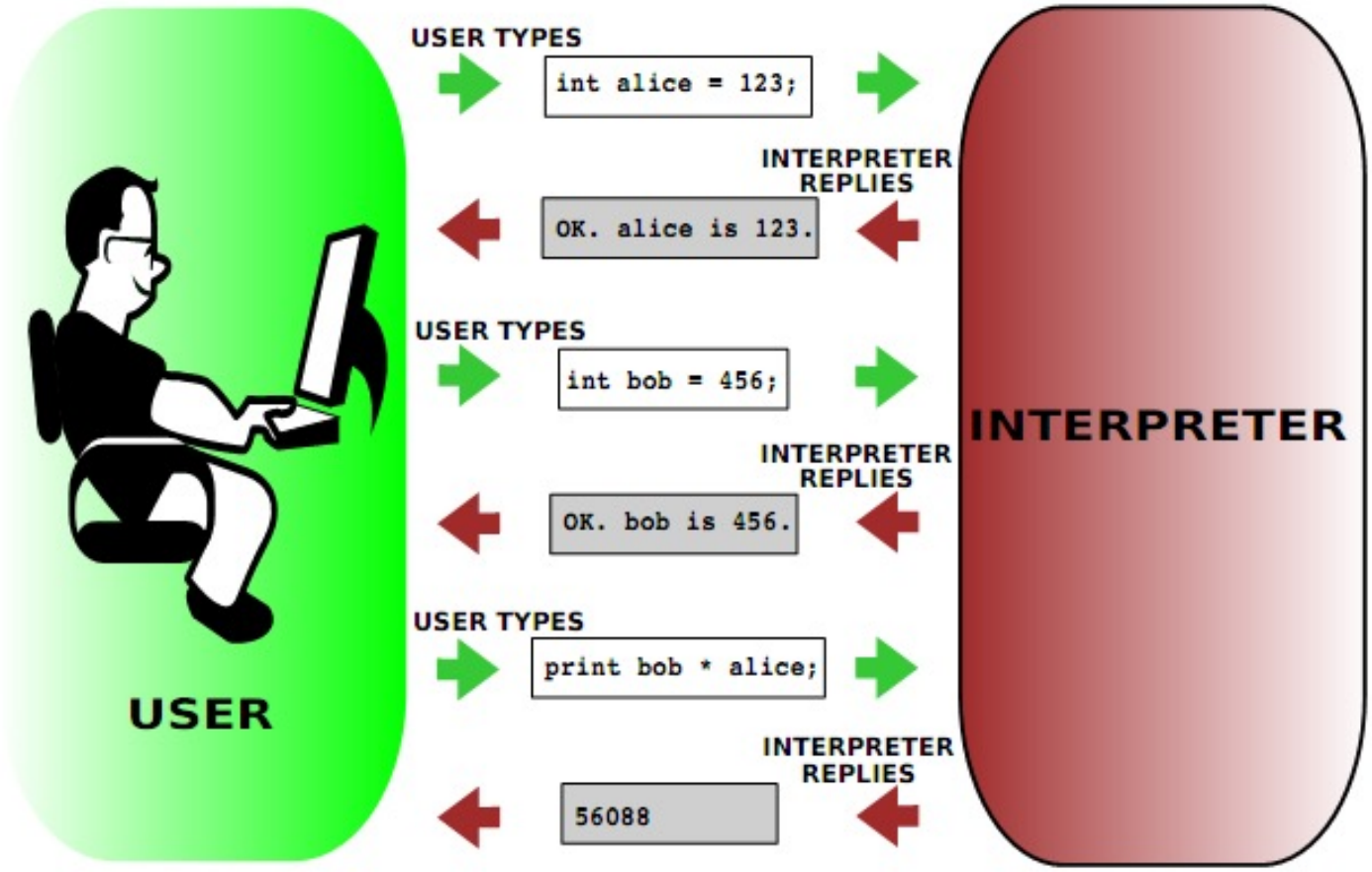
How are languages implemented



How are languages implemented

METU Computer Engineering

INTERPRETIVE APPROACH



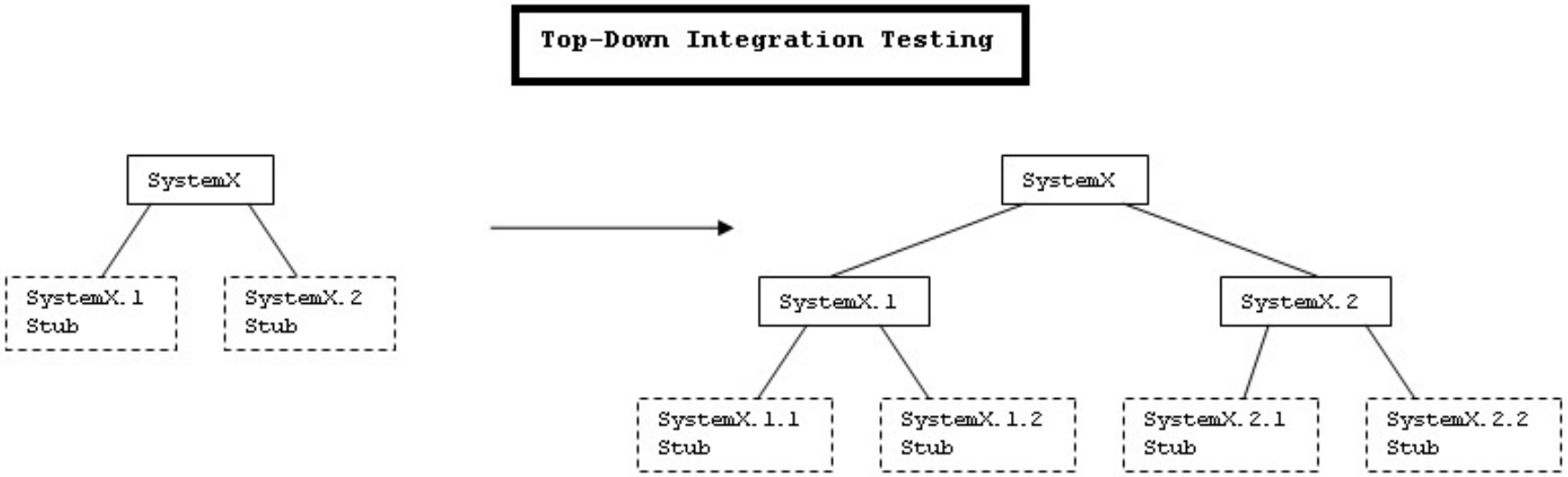


How is a program written?

- Modular & Functional Breakdown
- For example:
  - User interface module
  - Database module
  - Control module

Testing

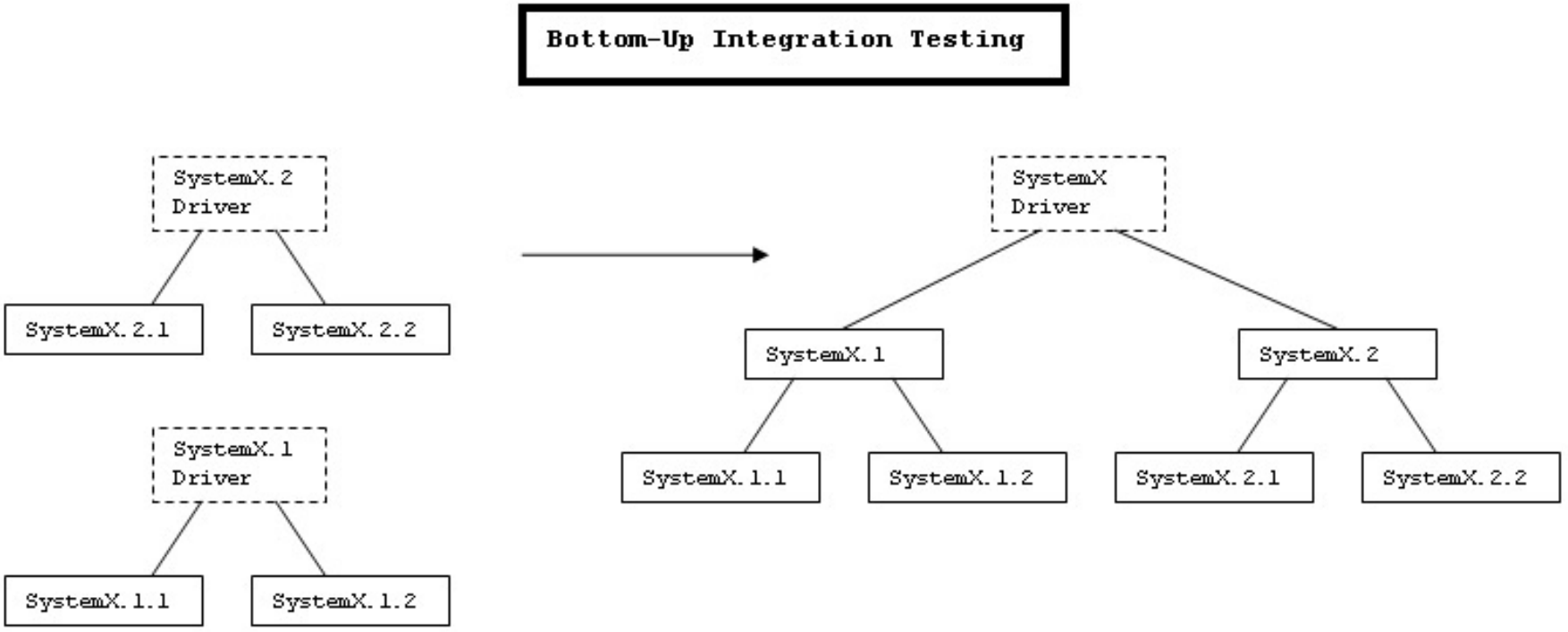
- Top-down Testing



http://sce.uhcl.edu/whiteta/sdp/subSystemIntegrationTesting.html

Testing

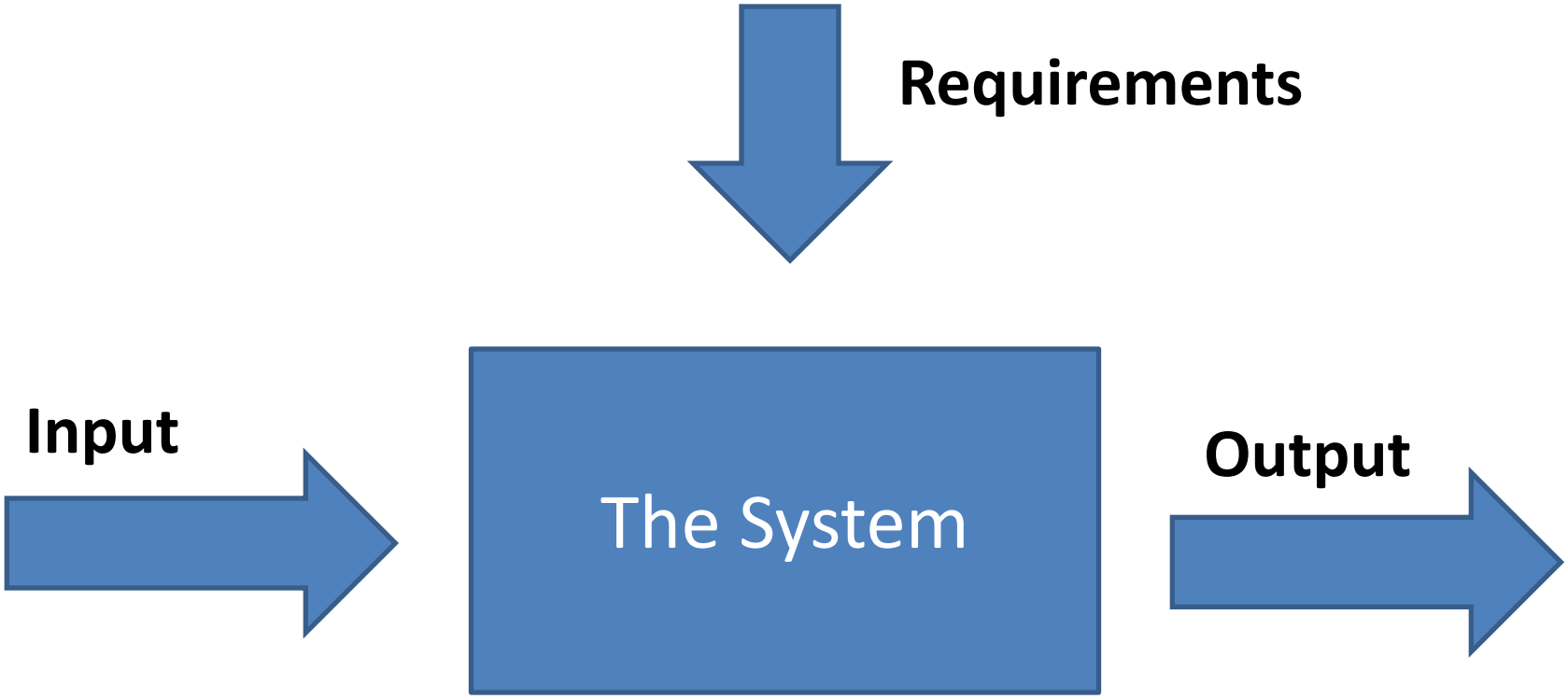
- Bottom-up Testing



http://sce.uhcl.edu/whiteta/sdp/subSystemIntegrationTesting.html

Testing

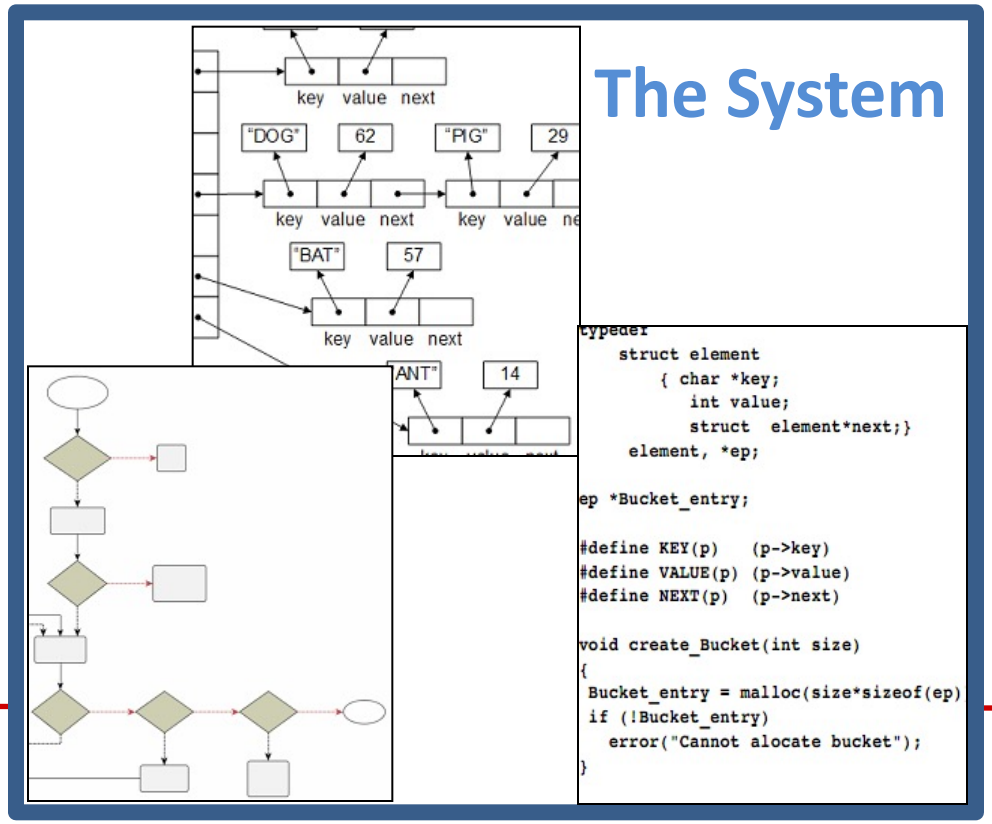
- Black-box Testing



Testing

- White-box Testing

Requirements



Input

Output



Bugs, Errors

METU Computer Engineering

- Syntax Errors

  Area = 3.1415 * R * R

  Area = 3.1415 x R x R

- Run-time Errors

```
>>> def SqrtDelta(a,b,c):
>>>       return sqrt(b*b - 4*a*c)
>>>
>>> print SqrtDelta(1,3,1)
2.236067977497898
>>> print SqrtDelta(1,1,1)
ValueError: math domain error
```



Bugs, Errors

METU Computer Engineering

- Logical Errors

$$root_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$



```
>>> root1 = (- b + sqrt(b*b - 4*a*c)) / 2*a
```

- Design Errors

$$x^3 + ax^2 + bx + c = 0$$

$$root_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$